

Relatório de Arquitetura

Data Lakehouse de E-commerce com Databricks, Delta Lake e Unity Catalog

Aluno	Luis Maio
Disciplina	Implementação de Data Lakehouse com Databricks e Delta Lake [26E3_3]
Instituição	Instituto Infnet — 2026
Repositório	github.com/LuisFelipeMTCS/ecommerce_lakehouse
Ambiente	Databricks Free Edition (computação serverless)

Sumário

- | | |
|--|--|
| 1. Introdução e escopo | 9. Streaming, materialized views e views comuns |
| 2. Arquitetura da solução | 10. Dependência externa e publicação dos datasets |
| 3. Esquema de transação do Delta Lake | 11. Manutenção e operação |
| 4. Ingestão escalável com Auto Loader | 12. Métricas do pipeline e impacto das otimizações |
| 5. Qualidade de dados: expectations e quarentena | 13. Adaptações técnicas durante a implementação |
| 6. Captura de mudanças (CDC) | 14. Limitações conhecidas |
| 7. Governança no Unity Catalog | 15. Atendimento aos requisitos |
| 8. Otimização e performance | 16. Conclusão |

Todas as consultas citadas estão no notebook `teste/validacao_rubricas.ipynb`, do repositório, e os números deste relatório foram lidos do workspace após a execução do pipeline. Quando um resultado depende de uma condição do ambiente, isso está dito no texto.

1. Introdução e escopo

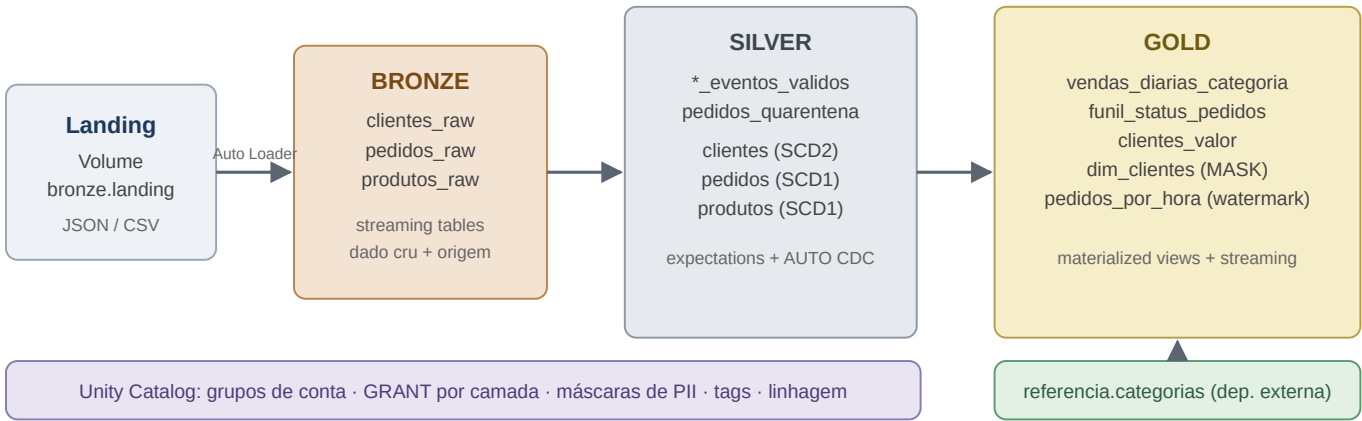
Este projeto implementa um Data Lakehouse para um cenário de e-commerce, no padrão Medallion (bronze, silver e gold), usando Lakeflow Declarative Pipelines (denominação atual do Delta Live Tables, DLT) em SQL, Auto Loader para ingestão, Delta Lake como formato de armazenamento e Unity Catalog como camada de governança. O projeto é empacotado como um Databricks Asset Bundle, o que permite publicar e executar todo o ambiente com comandos de CLI.

Como não há uma loja real, os dados são sintéticos e gerados por `src/setup/gerador_dados.py` em dois lotes. O gerador injeta defeitos de propósito (cerca de 3% nos clientes e 4% nos pedidos) para que as regras de qualidade tenham o que detectar, e o segundo lote traz atualizações, exclusões, mudanças de status e um evento fora de ordem para exercitar o CDC.

Entidade	Formato	Lote 1	Lote 2	Total na bronze
Clientes (eventos CDC)	JSON	500	71	571
Pedidos (eventos)	JSON	3.000	1.800	4.800
Produtos (snapshot com operação)	CSV	40	6	46

A execução ocorreu em um workspace real da Databricks Free Edition, que oferece apenas computação serverless. As consequências dessa restrição estão discutidas nas seções 8 e 14.

2. Arquitetura da solução



2.1 Camadas e arquivos

Camada	Arquivo	Responsabilidade
Bronze	<code>src/pipeline/bronze.sql</code>	Ingestão bruta via Auto Loader, schema fixado por <code>schemaHints</code> , coluna de dados resgatados, sem regra de negócio
Silver	<code>src/pipeline/silver.sql</code>	Validação com expectations, quarentena de pedidos e aplicação de CDC (SCD1 e SCD2)
Gold	<code>src/pipeline/gold.sql</code>	Agregações de negócio, dimensão com PII mascarada e streaming com watermark

Os schemas do catálogo `ecommerce_lakehouse` (Figura 1) separam as camadas e também o apoio: referencia (dependência externa), governança (funções de máscara e filtro), auditoria (snapshots e checkpoints) e monitoramento (views sobre o event log).

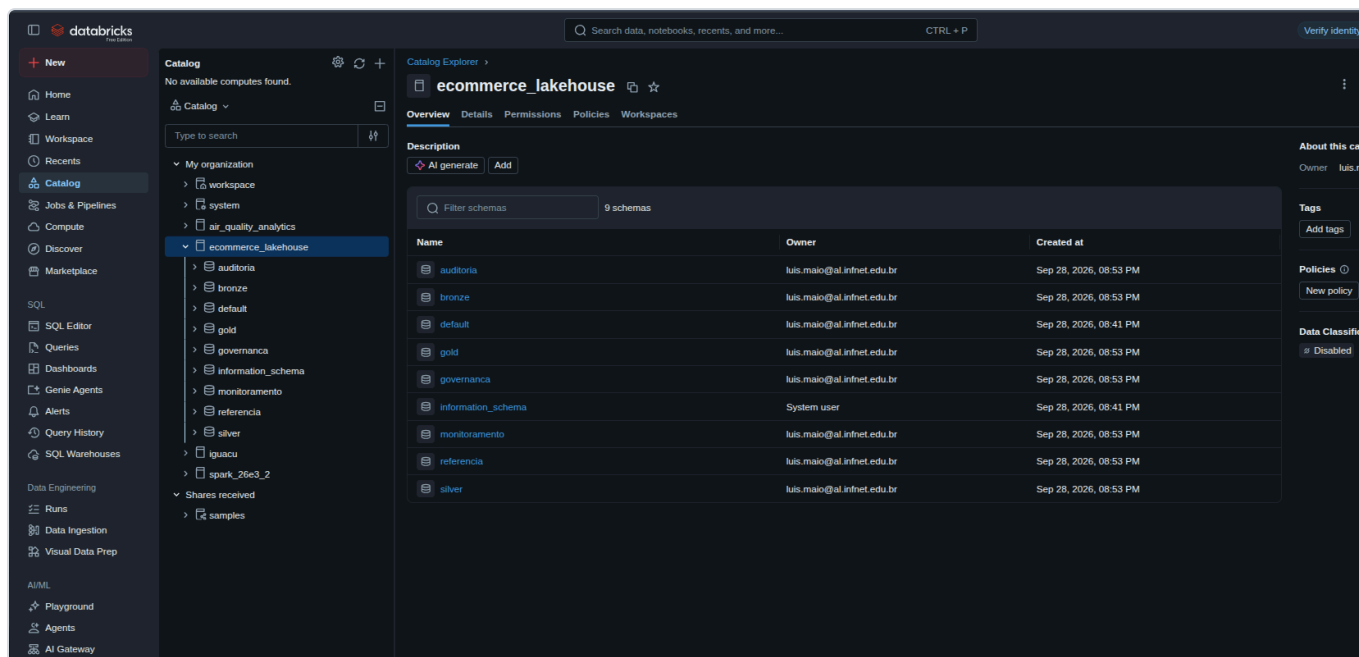


Figura 1 — Catálogo `ecommerce_lakehouse` e seus 9 schemas no Unity Catalog.

2.2 Orquestração com Delta Live Tables

O grafo de dependências `bronze → silver → gold` não é escrito pelo desenvolvedor: cada tabela declara o que contém (`CREATE OR REFRESH STREAMING TABLE`, `CREATE MATERIALIZED VIEW`) e o motor do Lakeflow infere a ordem, paraleliza o que é independente e executa apenas o necessário. O pipeline `ecommerce_medallion_dlt` tem 16 flows, todos concluídos na execução da Figura 2, com modo `continuous: false` (execução sob demanda) e computação `serverless`.

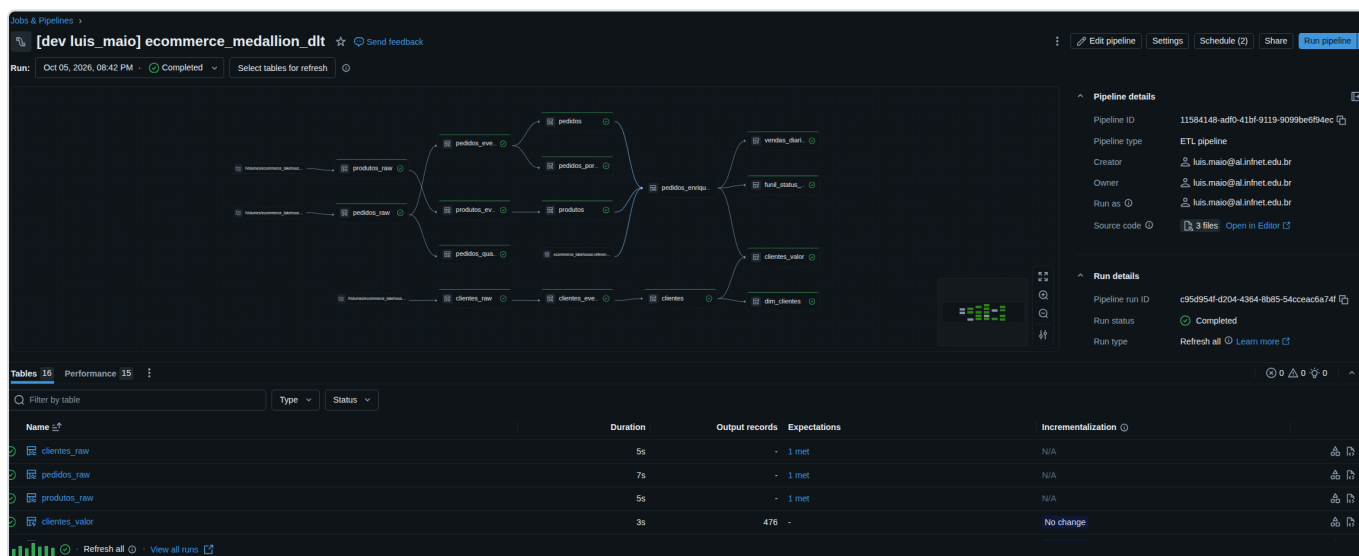


Figura 2 — Grafo do pipeline `ecommerce_medallion_dlt`: execução concluída, 16 tabelas e expectations por tabela.

2.3 Workflow e automação

O projeto é um Databricks Asset Bundle (`databricks.yml` e `resources/*.yml`). Três jobs definem a automação:

Job	Gatilho	Conteúdo
setup_e_carga	Manual (bundle run)	10 tarefas encadeadas: catálogo, governança, dados do lote 1, pipeline, dados do lote 2, pipeline, manutenção Delta, métricas, demo de checkpoint e validação de governança
ingestao	file_arrival no volume de landing, mínimo de 300 s entre disparos	Atualiza o pipeline e as métricas de qualidade quando chegam arquivos. Pausado
manutencao	Cron 0 30 2 * * ?, fuso America/Sao_Paulo	Manutenção noturna das tabelas Delta. Pausado

Os jobs de ingestão e manutenção existem e estão configurados, mas permanecem pausados para não consumir computação enquanto não há uso real. A Figura 3 mostra a execução completa e bem-sucedida do job principal.

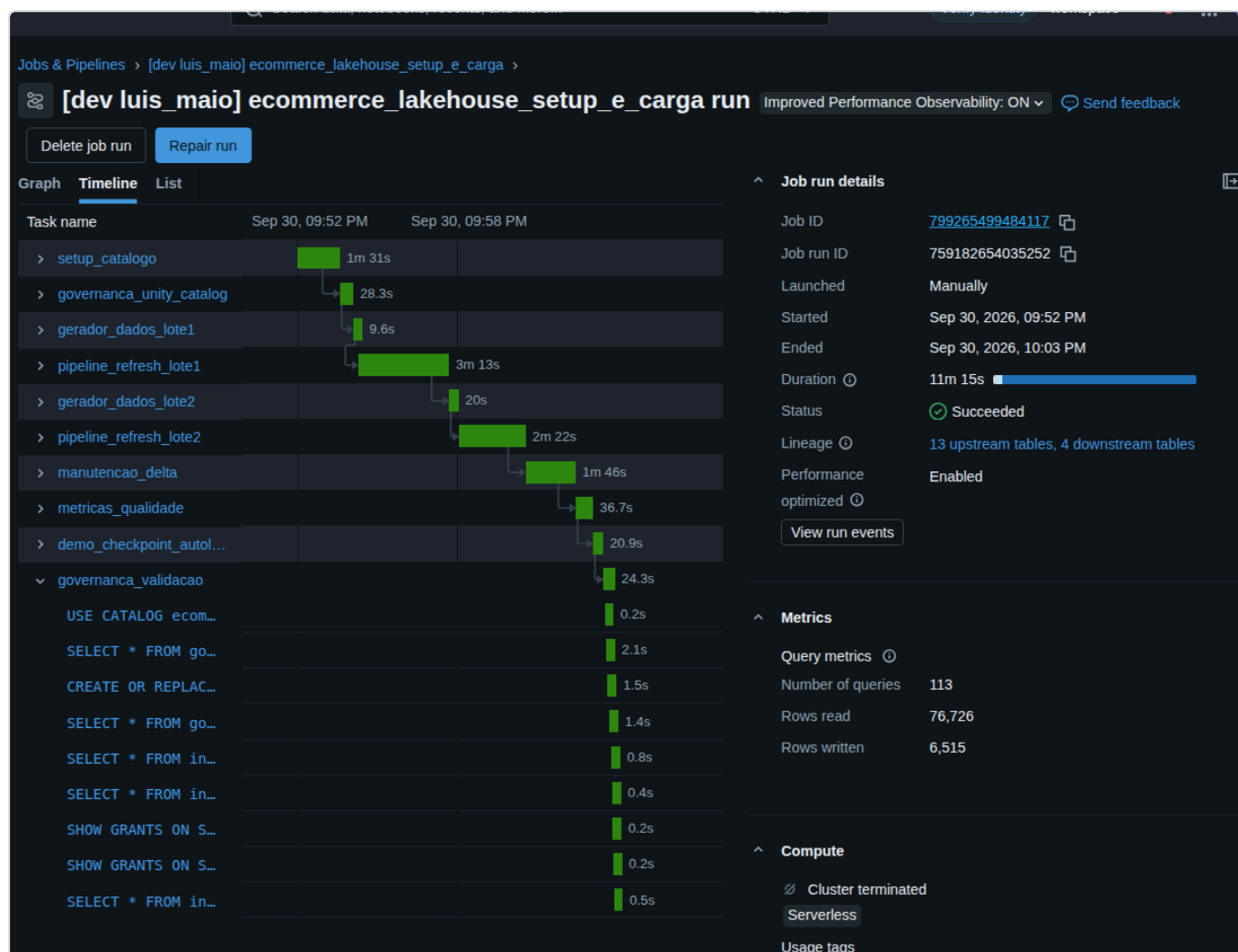


Figura 3 — Execução do job setup_e_carga: 10 tarefas em sequência, 11 min 15 s, 113 consultas, status Succeeded, computação serverless.

3. Esquema de transação do Delta Lake

Toda tabela do projeto é uma tabela Delta. O estado de cada uma é definido pelo seu *transaction log* (diretório `_delta_log`): uma sequência ordenada de commits atômicos, cada um descrevendo quais arquivos de dados

foram adicionados ou removidos e quais metadados mudaram. Um commit só passa a valer quando seu arquivo é gravado com sucesso no log, e é essa regra, e não o conteúdo dos arquivos Parquet, que garante atomicidade, consistência, isolamento e durabilidade. Cada commit gera uma versão imutável, e por isso é possível consultar ou restaurar qualquer estado anterior (time travel).

A anatomia da tabela e o protocolo aparecem na Figura 4 (formato, arquivos, localização) e na Figura 6 (versões mínimas de leitura e escrita e recursos habilitados: `minReaderVersion = 3`, `minWriterVersion = 7`, `deletionVectors` e `enableDeletionVectors = true`).

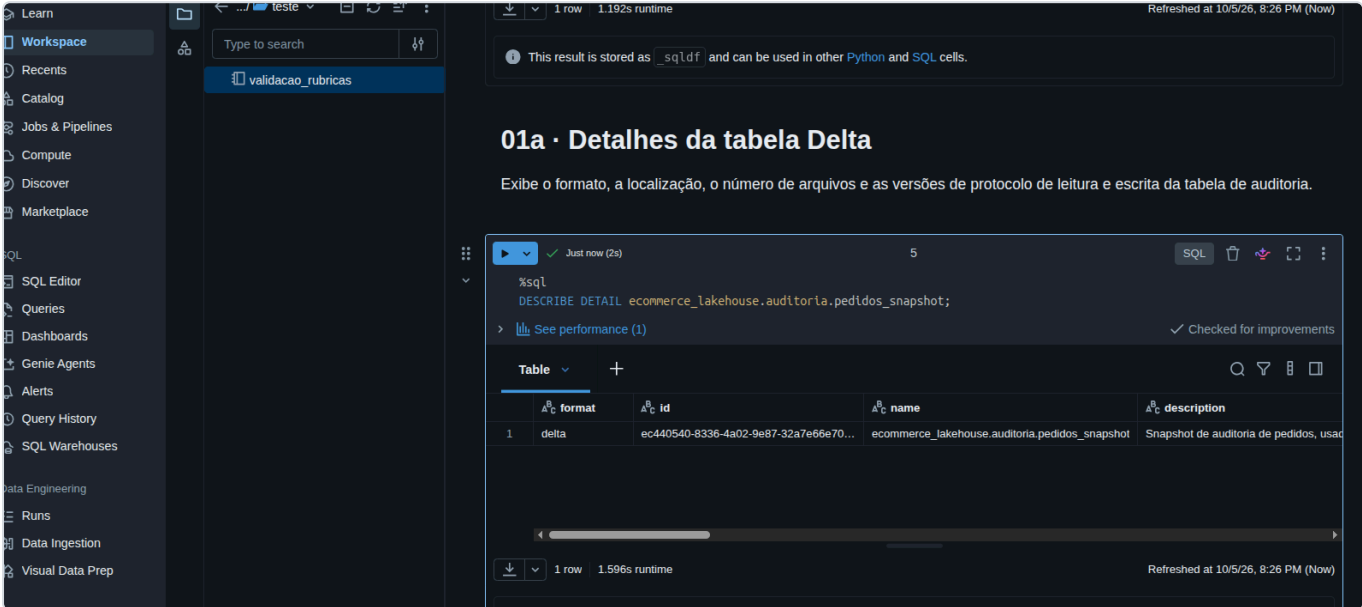


Figura 4 — DESCRIBE DETAIL da tabela auditoria.pedidos_snapshot: formato delta e metadados da tabela.

O notebook `src/operacao/manutencao_delta.sql` cria a tabela de auditoria `auditoria.pedidos_snapshot` e aplica, em sequência, operações que deixam rastro no log. As métricas abaixo foram extraídas do próprio histórico (`operationMetrics`):

Versão	Operação	Efeito registrado no log
0	CREATE TABLE AS SELECT	Snapshot de <code>silver.pedidos</code> : 3.873 linhas, 1 arquivo (50.971 bytes)
1	UPDATE	176 linhas atualizadas; um deletion vector adicionado, sem reescrever arquivo
2	DELETE	22 linhas removidas via deletion vector
3	MERGE	176 linhas atualizadas e 22 inseridas, reconciliando com a silver
4	DELETE	"Acidente" proposital: 3.873 linhas excluídas (2 arquivos, 56.389 bytes removidos)
5	RESTORE	Estado anterior recuperado por time travel: 2 arquivos restaurados
6	OPTIMIZE ZORDER BY (cliente_id, pedido_ts)	2 arquivos (56.389 bytes) compactados em 1 (51.125 bytes)
7 e 8	VACUUM START / END	Limpeza com retenção de 168 horas; nenhum arquivo elegível na ocasião

As versões seguintes (9 em diante) correspondem às demais execuções do job de carga, que repete o ciclo sobre a mesma tabela.

01b · Histórico de transações

Lista cada commit do log de transações da tabela, com a operação executada, seus parâmetros e suas métricas.

▶

▼

✓

Just now (13s)

7

SQL

🗑️

🔄

🔗

⋮

▼

%sql

SELECT version, timestamp, operation, operationParameters, operationMetrics

FROM (DESCRIBE HISTORY ecommerce_lakehouse.auditoria.pedidos_snapshot)

ORDER BY version;

> [See performance \(1\)](#)

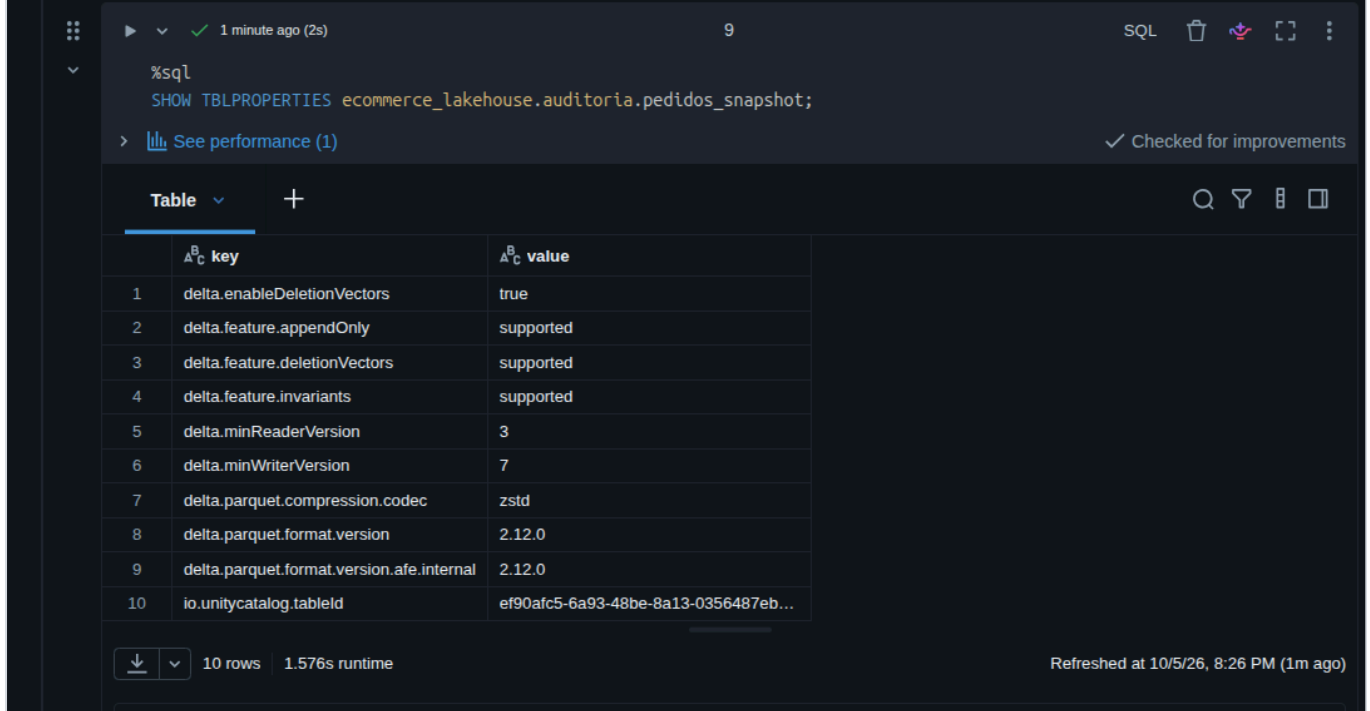
✓ Checked for improvements

	version	timestamp	operation	operationParameters
1	0	2026-09-29T02:24:49.000+00:...	CREATE TABLE AS SELECT	> {"partitionBy":[""],"clusterBy":[""],"description":"Snapshot de audit
2	1	2026-09-29T02:24:56.000+00:...	UPDATE	> {"predicate":["\"((status#15935 = ENVIADO) AND (pedido_ts#159
3	2	2026-09-29T02:24:59.000+00:...	DELETE	> {"predicate":["\"((status#16465 = CANCELADO) AND (pedido_ts#
4	3	2026-09-29T02:25:05.000+00:...	MERGE	> {"predicate":["\"(pedido_id#16693L = pedido_id#16717L)\"]","clus
5	4	2026-09-29T02:25:10.000+00:...	DELETE	> {"predicate":["\"true\""]}
6	5	2026-09-29T02:25:20.000+00:...	RESTORE	> {"version":"3","timestamp":null}
7	6	2026-09-29T02:25:34.000+00:...	OPTIMIZE	> {"clusterBy":[""],"zOrderBy":["\"cliente_id\"\",\"pedido_ts\""],"batchlc
8	7	2026-09-29T02:25:59.000+00:...	VACUUM START	> {"retentionCheckEnabled":"true","defaultRetentionMillis":"604800
9	8	2026-09-29T02:25:59.001+00:...	VACUUM END	> {"status":"COMPLETED"}
10	9	2026-09-29T02:34:48.000+00:...	UPDATE	> {"predicate":["\"((status#24420 = ENVIADO) AND (pedido_ts#244

Figura 5 — DESCRIBE HISTORY: UPDATE, DELETE, MERGE, DELETE, RESTORE, OPTIMIZE e VACUUM, uma linha por commit.

01c · Propriedades da tabela

Mostra as propriedades e os recursos do Delta habilitados na tabela, como os vetores de exclusão.



The screenshot shows a Databricks SQL query editor with the following SQL command:

```
%sql
SHOW TBLPROPERTIES ecommerce_lakehouse.auditoria.pedidos_snapshot;
```

Below the query, a table displays the properties of the Delta table. The table has two columns: 'key' and 'value'.

	key	value
1	delta.enableDeletionVectors	true
2	delta.feature.appendOnly	supported
3	delta.feature.deletionVectors	supported
4	delta.feature.invariants	supported
5	delta.minReaderVersion	3
6	delta.minWriterVersion	7
7	delta.parquet.compression.codec	zstd
8	delta.parquet.format.version	2.12.0
9	delta.parquet.format.version.afe.internal	2.12.0
10	io.univocitycatalog.tableId	ef90afc5-6a93-48be-8a13-0356487eb...

At the bottom of the table, it indicates '10 rows' and '1.576s runtime'. The interface also shows a 'Refreshed at 10/5/26, 8:26 PM (1m ago)' timestamp.

Figura 6 — Propriedades e recursos do protocolo Delta habilitados na tabela.

4. Ingestão escalável com Auto Loader

O gerador grava os arquivos em um Volume do Unity Catalog (`/Volumes/ecommerce_lakehouse/bronze/landing`), com uma pasta por entidade. As três streaming tables da bronze leem essas pastas com `STREAM read_files(...)`, o Auto Loader declarativo, que mantém o registro dos arquivos já processados e lê somente os novos, sem reprocessar o histórico.

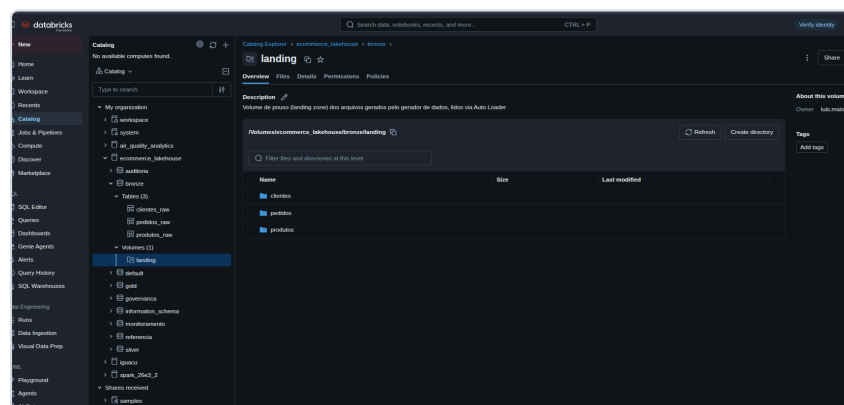


Figura 7 — Volume bronze.landing: uma pasta por entidade (clientes, pedidos, produtos).

Cada tabela fixa os tipos com `schemaHints`, guarda `arquivo_origem` (`_metadata.file_path`) e `data_ingestao`, e envia ao `_rescued_data` qualquer campo fora do schema esperado, monitorado por uma expectation de aviso. A Figura 8 mostra o efeito incremental: cada arquivo, nos dois lotes, aparece lido uma

única vez, com a contagem esperada (500 + 71 clientes, 3.000 + 1.800 pedidos, 40 + 6 produtos), e nenhuma linha caiu em `_rescued_data`.

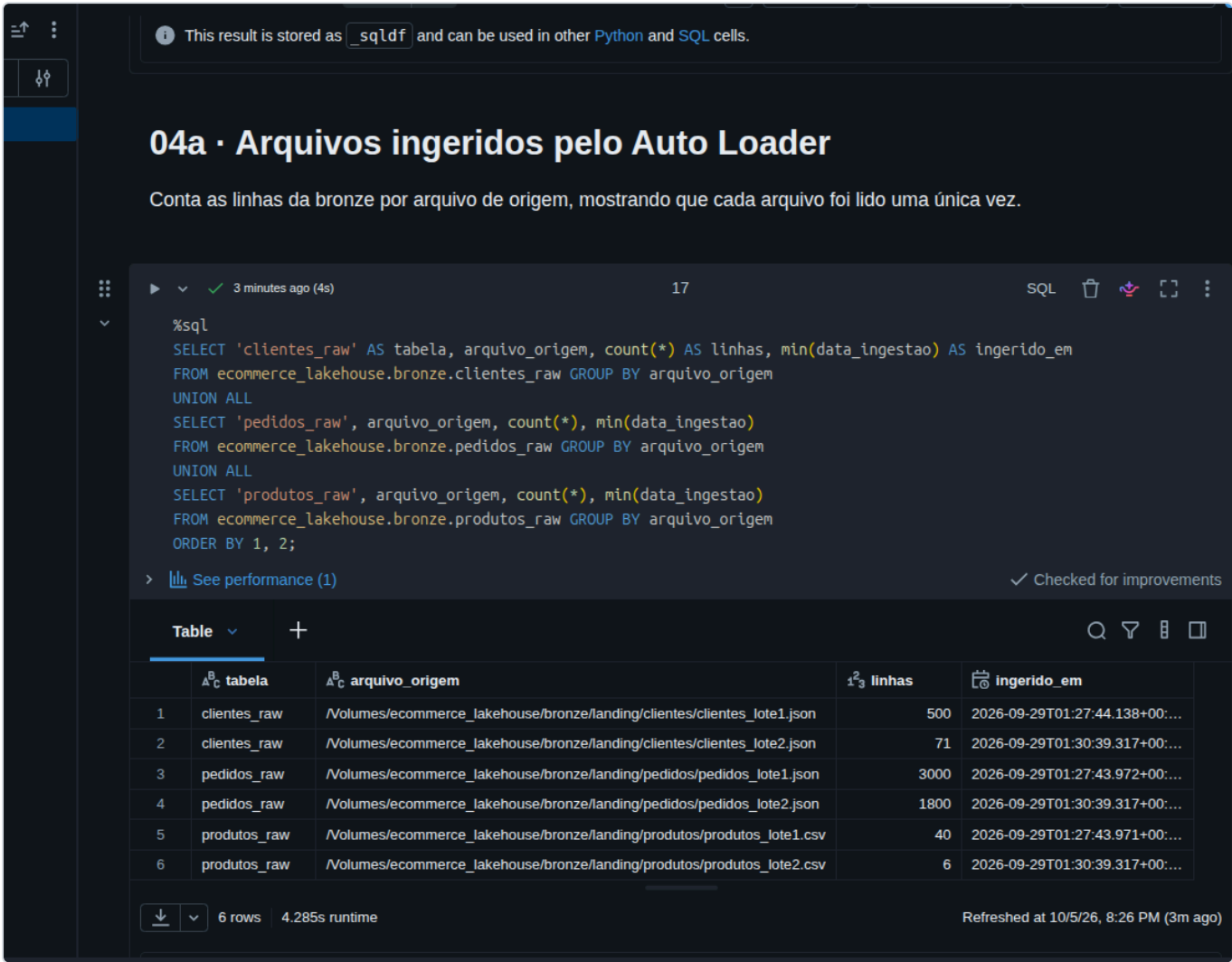


Figura 8 — Linhas da bronze por arquivo de origem: lote 1 e lote 2 de cada entidade.

As tabelas da bronze também habilitam compactação automática (`autoOptimize.autoCompact` e `optimizeWrite`) e bloqueiam a recriação acidental dos dados com `pipelines.reset.allowed = false`, já que a bronze é a única cópia fiel do que chegou.

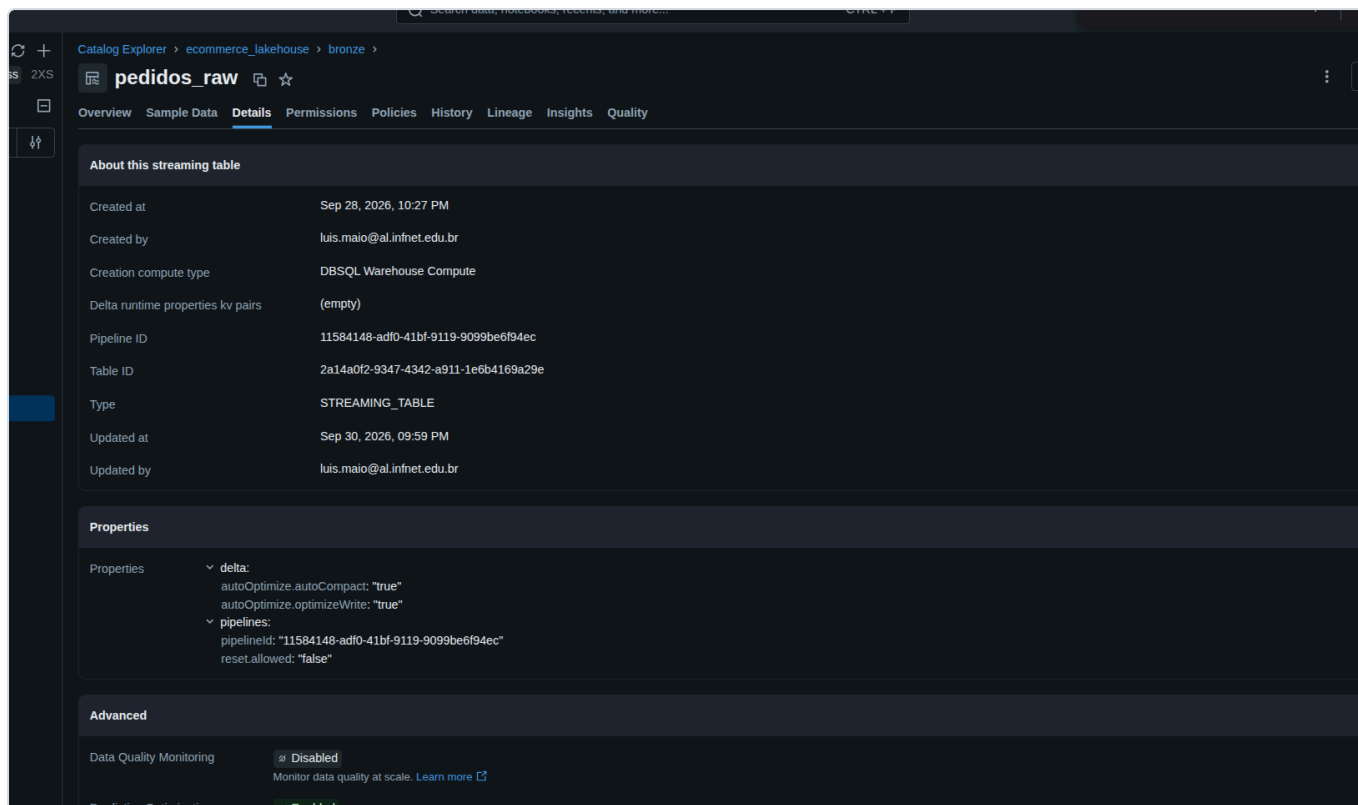


Figura 9 — Propriedades de bronze.pedidos_raw: autoCompact, optimizeWrite e reset.allowed = false.

5. Qualidade de dados: expectations e quarentena

A silver valida os eventos em streaming tables intermediárias (`*_eventos_validos`) antes de aplicar qualquer CDC. Foram usadas as três severidades do Lakeflow:

Severidade	Cláusula	Uso no projeto
Aviso	EXPECT (. . .)	Conta a violação sem descartar: UF ausente em clientes, canal ausente em pedidos, <code>_rescued_data</code> na bronze
Descarte	ON VIOLATION DROP ROW	Remove do fluxo principal: CPF ou e-mail inválido, pedido sem identificador, quantidade ou preço inválidos, status inválido
Falha	ON VIOLATION FAIL UPDATE	Interrompe a atualização: produto sem <code>produto_id</code> (a chave do CDC) e receita negativa na gold

5.1 Resultado por regra

Os totais abaixo vêm do event log do pipeline, acumulados nas atualizações registradas:

Dataset	Regra	Aprovados	Reprovados
silver.clientes_eventos_validos	cpf_valido	563	8
silver.clientes_eventos_validos	email_valido	562	9
silver.clientes_eventos_validos	id_valido, sequencia_valida, operacao_valida, uf_valida	571	0
silver.pedidos_eventos_validos	pedido_id_valido	4.767	33
silver.pedidos_eventos_validos	cliente_id_valido	4.767	33

Regra violada	Pedidos
quantidade_invalida	35
pedido_id_nulo	33
cliente_id_nulo	33
preco_invalido	31
status_invalido	22
Total	154 (3,2% dos 4.800 eventos)

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

05b · Quarentena por regra

Conta os pedidos rejeitados na quarentena, agrupados pela regra violada.

```
%sql
SELECT regra, count(*) AS qtd
FROM ecommerce_lakehouse.silver.pedidos_quarentena
LATERAL VIEW explode(regras_violadas) AS regra
GROUP BY regra
ORDER BY qtd DESC;
```

> [See performance \(1\)](#) ✓ Checked for improvements

	regra	qtd
1	quantidade_invali...	35
2	pedido_id_nulo	33
3	cliente_id_nulo	33
4	preco_invalido	31
5	status_invalido	22

5 rows 2.278s runtime Refreshed at 10/5/26, 8:26 PM (4m ago)

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

05c · Amostra da quarentena

Figura 11 — Pedidos em quarentena por regra violada.

Observação. O gerador também produz pedidos com "data futura", mas a regra `data_coerente` compara `pedido_ts` com `evento_ts`, e o gerador grava o mesmo valor nos dois campos. Por isso a regra não reprova nenhum pedido (0 de 4.800) e a causa "data futura" não aparece na quarentena. Ver seção 14.

6. Captura de mudanças (CDC)

Cada entidade usa `CREATE FLOW ... AS AUTO CDC INTO` sobre a streaming table de eventos validados, com a estratégia de SCD adequada ao uso analítico:

Entidade	Estratégia	Chave / ordenação	Motivo
Clientes	SCD Tipo 2 (histórico)	cliente_id / evento_seq	Preservar mudanças de dados cadastrais ao longo do tempo
Pedidos	SCD Tipo 1 (estado atual)	pedido_id / evento_ts	Interessa o status mais recente do pedido
Produtos	SCD Tipo 1 com DELETE	produto_id / atualizado_em	Produtos descontinuados são removidos, não versionados

6.1 Clientes (SCD2)

Cada versão tem `__START_AT` e `__END_AT`; a versão vigente tem `__END_AT` nulo. Um cliente que mudou de e-mail e telefone no lote 2 aparece com duas linhas, a antiga fechada no momento da mudança (Figura 12).

06a · Clientes com histórico (SCD2)

Lista os clientes que têm mais de uma versão, mostrando o início e o fim de cada uma. A versão atual tem `__END_AT` nulo.

```

%sql
SELECT cliente_id, email, telefone, __START_AT, __END_AT
FROM ecommerce_lakehouse.silver.clientes
WHERE cliente_id IN (
  SELECT cliente_id FROM ecommerce_lakehouse.silver.clientes
  GROUP BY cliente_id HAVING count(*) > 1
)
ORDER BY cliente_id, __START_AT
LIMIT 20;

```

	cliente_id	email	telefone	__START_AT	__END_AT
1	2	bruno.fernandes2@example.com	11354319726	2	102029
2	2	nov0.9338.bruno.fernandes2@example.com	11327504504	102029	null
3	6	karina.lima6@example.com	11791099629	6	102041
4	6	nov0.1447.karina.lima6@example.com	11377140740	102041	null
5	8	joão.almeida8@example.com	11063847079	8	102008
6	8	nov0.9519.joão.almeida8@example.com	11376709597	102008	null
7	18	lucas.pereira18@example.com	11629431652	18	102004

Figura 12 — Clientes com mais de uma versão: a versão antiga com `__END_AT` preenchido, a atual com `__END_AT` nulo.

Os números fecham com as contas do pipeline:

- 571 eventos recebidos; 17 reprovados (8 CPF + 9 e-mail) → **554 eventos válidos**.
- Os 10 eventos DELETE do lote 2 fecham a versão vigente e não criam linha nova → $554 - 10 = 544$ **versões** na tabela.
- Dessas, **476 são vigentes** e **68 são históricas**, para 486 clientes distintos (dos 500 enviados, 14 foram rejeitados já na inserção). Os 10 clientes excluídos ficam sem versão vigente.

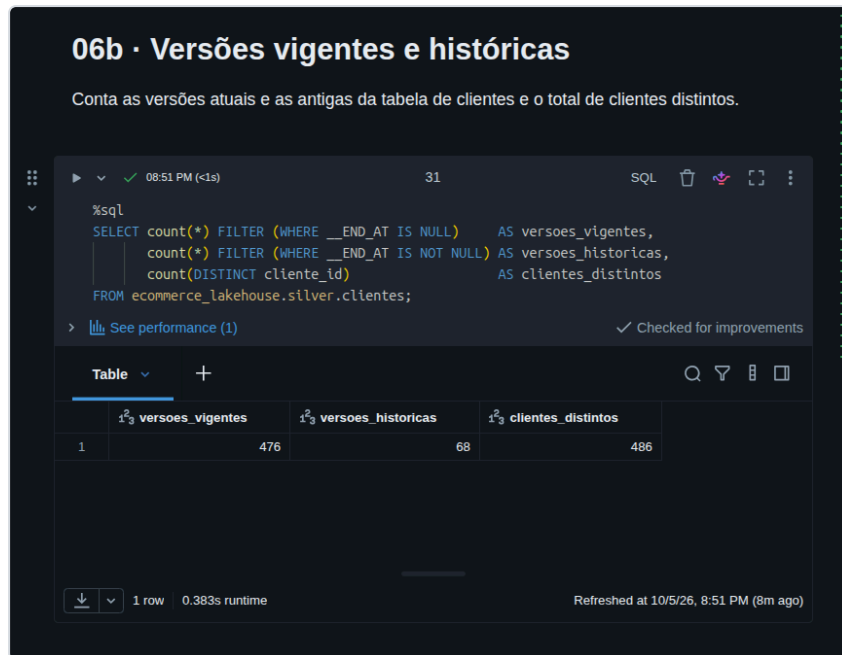


Figura 13 — Versões vigentes (476), históricas (68) e clientes distintos (486).

6.2 Evento fora de ordem

O lote 2 inclui, de propósito, um `UPDATE` do cliente 200 com `evento_seq = 1`, anterior ao `INSERT` original (`evento_seq = 200`). O CDC usa `SEQUENCE BY` para ordenar os eventos, e o resultado em SCD2 foi:

cliente_id	email	__START_AT	__END_AT
200	evento.atrasado.ignorar@example.com	1	200
200	bruno.lima200@example.com	200	102000
200	novo.5640.bruno.lima200@example.com	102000	(vigente)

O evento atrasado não foi descartado: foi encaixado cronologicamente no histórico, como uma versão anterior à inserção original, e **não alterou a versão vigente**. É o comportamento correto para uma dimensão SCD2, que reconstrói a linha do tempo pela chave de ordenação, e não pela ordem de chegada.

6.3 Pedidos e produtos (SCD1)

Em `silver.pedidos` há uma linha por pedido (3.873 linhas e 3.873 identificadores distintos): as 800 mudanças de status do lote 2 atualizam a linha existente. Em `silver.produtos` restam 39 dos 40 produtos, pois o `DELETE` remove fisicamente o registro. O resultado de negócio na gold (`gold.funil_status_pedidos`):

Status	Pedidos	Receita (R\$)
CRIADO	3.076	3.783.473,05
PAGO	211	294.496,91
ENTREGUE	202	257.295,36
CANCELADO	196	269.231,66
ENVIADO	188	204.031,84

7. Governança no Unity Catalog

7.1 Identidades e catalogação centralizada

Todos os dados estão no catálogo único `ecommerce_lakehouse` do Unity Catalog. O acesso é dado a três grupos **de conta** (não de workspace), pois o Unity Catalog e a função `is_account_group_member` só reconhecem esse nível de grupo:

Grupo	Acesso concedido
ecom_engenharia	Leitura e escrita em bronze, silver e gold e no volume de landing; vê PII real
ecom_pii_leitores	Leitura da gold com PII real
ecom_analistas	Leitura da gold com PII mascarada

O isolamento entre camadas é feito por `GRANT`: analistas e leitores de PII recebem `USE SCHEMA` e `SELECT` somente na gold, sem privilégio algum sobre bronze e silver, onde a PII completa está armazenada.

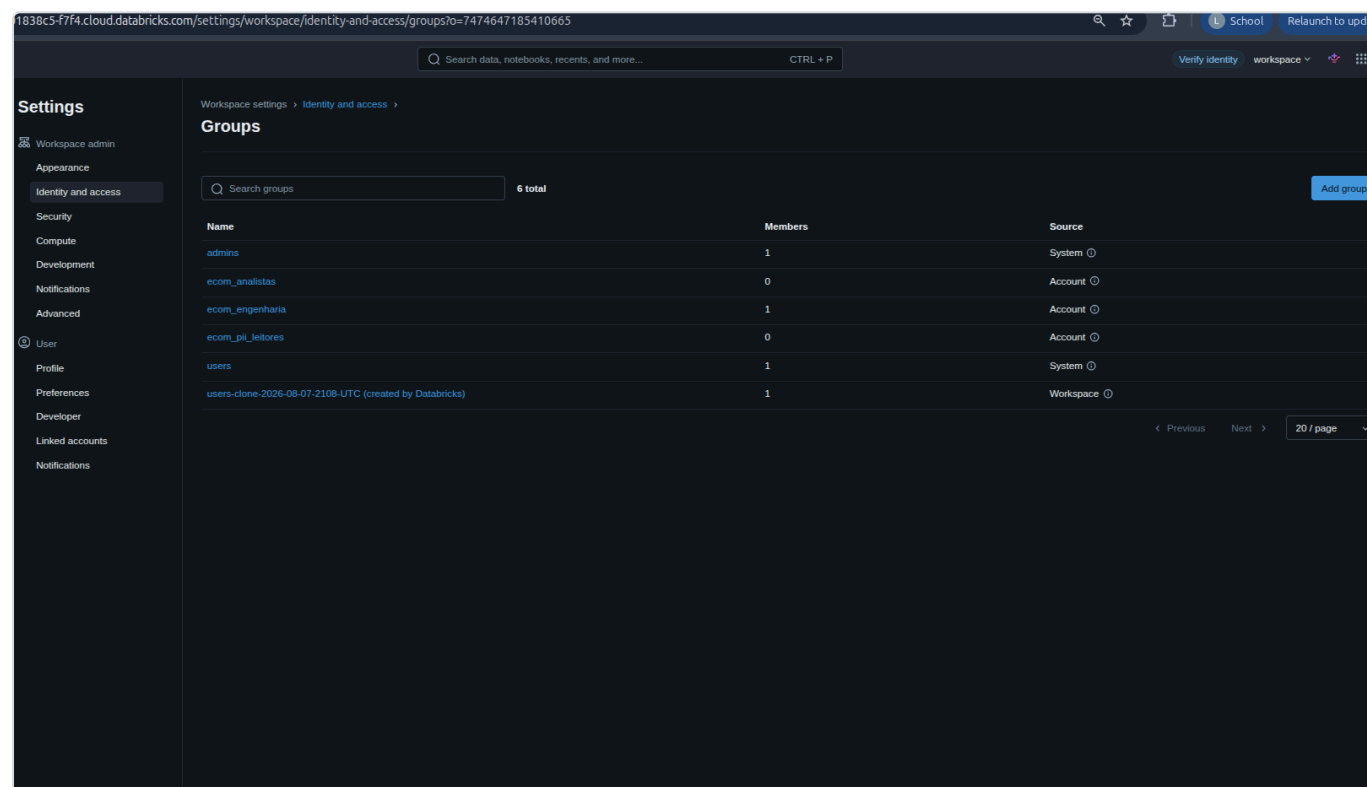


Figura 14 — Grupos do projeto (origem Account) em Settings > Identity and access > Groups.



Figura 15 — Verificação da identidade de execução: `is_account_group_member` para cada grupo do projeto.

7.2 Mascaramento de dados sensíveis

Três funções SQL no schema `governanca` (`mascara_email`, `mascara_cpf`, `mascara_telefone`) consultam `is_account_group_member(...)`: devolvem o valor real a `ecom_engenharia` e `ecom_pii_leitores` e um valor mascarado a qualquer outro leitor (por exemplo `***@example.com`, `***.*.*.*20`, `****0917`). As funções são ligadas às colunas `email`, `cpf` e `telefone` da materialized view `gold.dim_clientes` por meio de MASK.

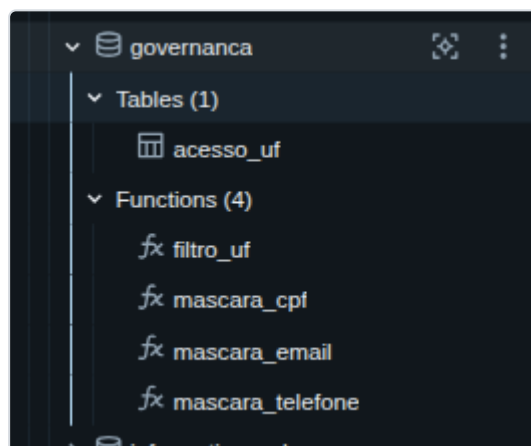


Figura 16 — Objetos do schema `governanca`: tabela `acesso_uf` e 4 funções (`filtro_uf` e as 3 máscaras).

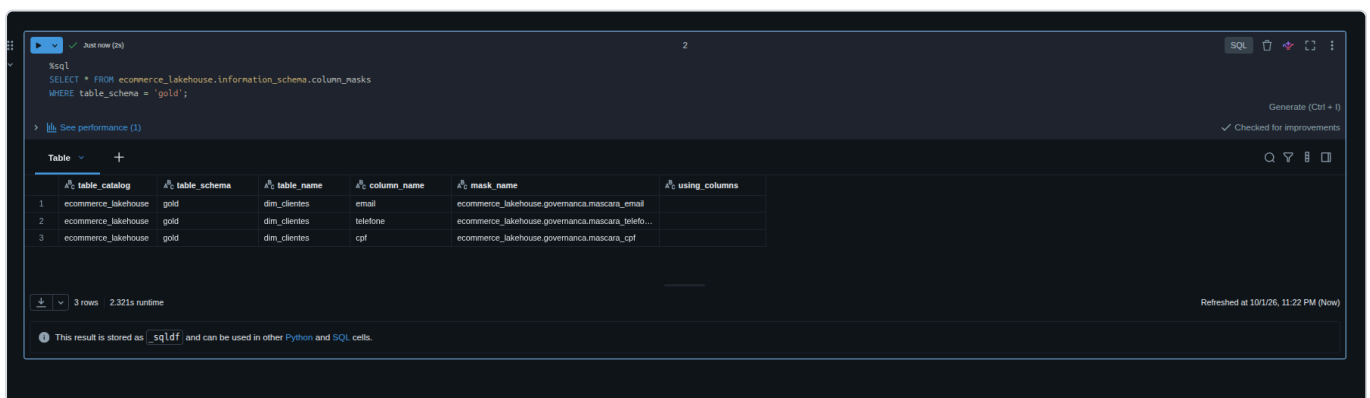


Figura 17 — `information_schema.column_masks`: as três colunas de `gold.dim_clientes` ligadas às funções de máscara.

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

```

%sql
SELECT cliente_id, nome, email, cpf, telefone, uf
FROM ecommerce_lakehouse.gold.dim_clientes LIMIT 10;

```

See performance (1) Checked for improvements

	cliente_id	nome	email	cpf	telefone	uf
1	3	Quesia Araújo	***@example.c...	***.***.20	****0917	AM
2	14	Sofia Ribeiro	***@example.c...	***.***.86	****2340	AM
3	15	Gabriela Santos	***@example.c...	***.***.40	****3429	AM
4	29	Carla Fernandes	***@example.c...	***.***.75	****0297	AM
5	45	Thiago Araújo	***@example.c...	***.***.26	****2287	AM
6	53	Carla Oliveira	***@example.c...	***.***.66	****4884	AM
7	193	Isabela Pereira	***@example.c...	***.***.92	****3479	AM
8	194	Gabriela Fernandes	***@example.c...	***.***.20	****5636	AM
9	213	Nicolas Rodrigues	***@example.c...	***.***.66	****4190	AM
10	244	Heitor Santos	***@example.c...	***.***.78	****7256	AM

10 rows 4.099s runtime Refreshed at 10/1/26, 11:30 PM (Now)

This result is stored as `_sqlidf` and can be used in other Python and SQL cells.

Figura 18 — Consulta a gold.dim_clientes com a máscara em efeito: e-mail, CPF e telefone mascarados.

7.3 Isolamento por região

O isolamento por UF foi desenhado como ROW FILTER nativo na própria materialized view. Na execução, o motor do Lakeflow desta versão rejeitou a cláusula em MATERIALIZED VIEW (PARSE_SYNTAX_ERROR próximo de FILTER). A solução foi aplicar o filtro numa view comum, gold.vw_clientes_regional, que reaplica as máscaras e usa governanca.filtro_uf(uf) como predicado, consultando a tabela governanca.acesso_uf (SP, RJ, MG, GO e DF para ecom_analistas). O limite dessa abordagem está na seção 14.

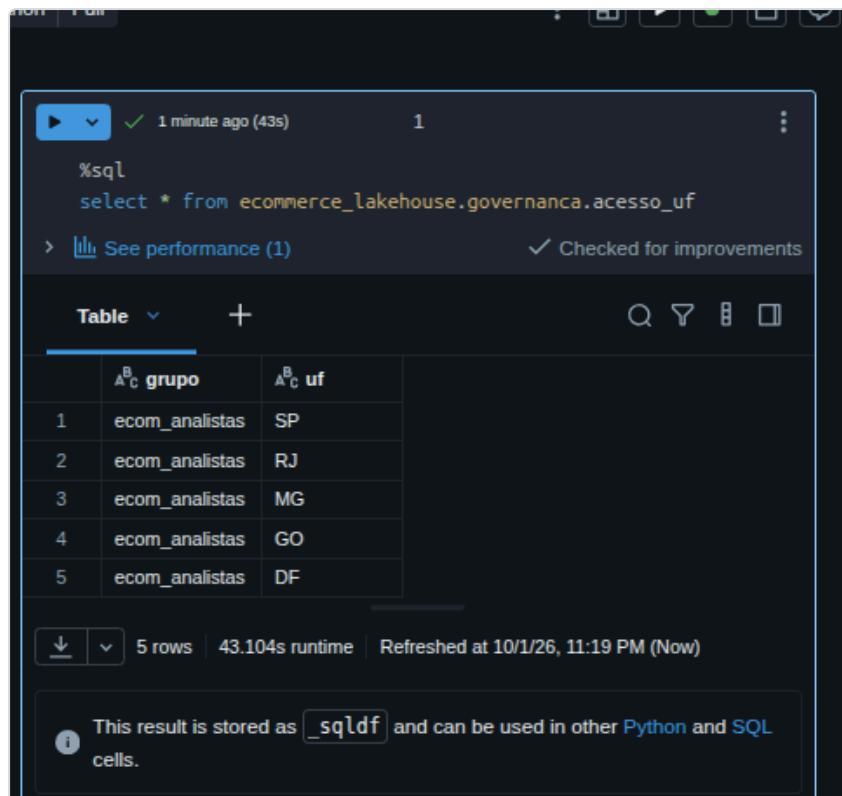


Figura 19 — Tabela governanca.acesso_uf: UFs liberadas para o grupo ecom_analistas.

7.4 Linhagem de dados

O Unity Catalog registra a linhagem automaticamente em `system.access.table_lineage` e na aba Lineage de cada tabela. A Figura 20 mostra a de `gold.dim_clientes` (origem `silver.clientes`, produzida pelo pipeline, e consumo pela view regional). A consulta ao sistema confirma a cadeia completa:

Origem	Destino
bronze.clientes_raw	silver.clientes_eventos_validos → silver.clientes
bronze.pedidos_raw	silver.pedidos_eventos_validos → silver.pedidos; silver.pedidos_quarentena
bronze.produtos_raw	silver.produtos_eventos_validos → silver.produtos
silver.pedidos_eventos_validos	gold.pedidos_por_hora
silver.pedidos, silver.produtos, referencia.categorias	gold.vendas_diarias_categoria, gold.funil_status_pedidos, gold.clientes_valor
silver.clientes	gold.dim_clientes → gold.vw_clientes_regional
silver.pedidos	auditoria.pedidos_snapshot
event log do pipeline	monitoramento.vw_event_log → monitoramento.vw_qualidade_expectations

Nenhuma view de negócio da gold lê diretamente a bronze: toda leitura passa pela validação e pelo CDC da silver. A dependência externa `referencia.categorias` aparece como origem sem ancestral dentro do pipeline.

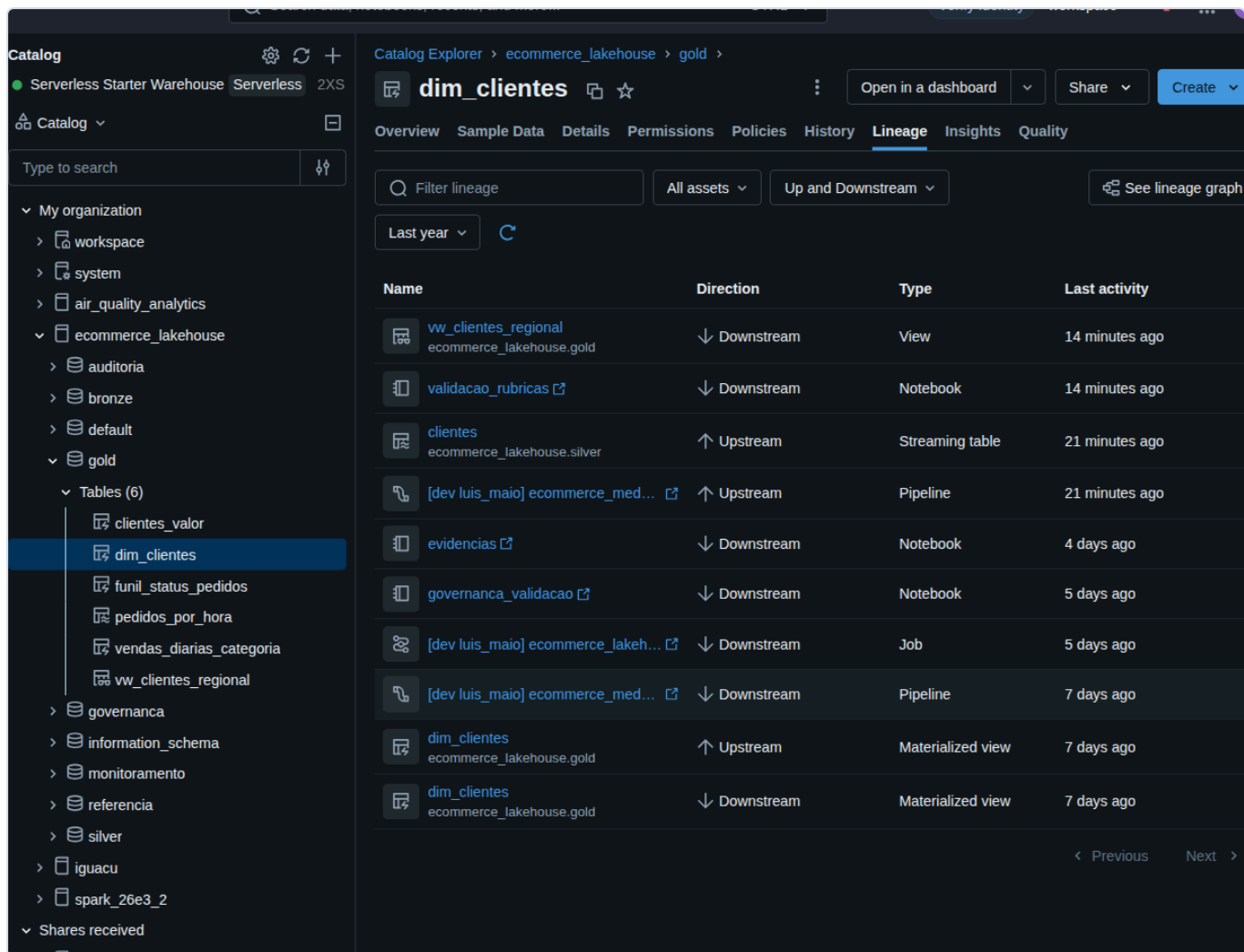


Figura 20 — Aba Lineage de gold.dim_clientes: origem silver.clientes (streaming table) e consumidores a jusante.

8. Otimização e performance

8.1 Z-Order, liquid clustering e deletion vectors

- **Z-Order.** `silver.clientes` (por `cliente_id`) e `silver.pedidos` (por `cliente_id`, `pedido_ts`) declaram `pipelines.autoOptimize.zOrderCols`, aplicado pela manutenção automática do Lakeflow. Na tabela de auditoria, o Z-Order foi executado de forma explícita (`OPTIMIZE ... ZORDER BY (cliente_id, pedido_ts)`), e o histórico (Figura 21) registra as colunas usadas e a compactação, 2 arquivos em 1, em cada execução.
- **Liquid clustering.** As materialized views da gold usam `CLUSTER BY` (por exemplo `data_pedido`, `categoria_id` em `vendas_diarias_categoria`), que dispensa reescrever o Z-Order a cada novo padrão de consulta.
- **Deletion vectors.** Habilitados nas tabelas de CDC e na tabela de auditoria: um `DELETE`, `UPDATE` ou `MERGE` marca linhas como removidas sem reescrever o arquivo Parquet inteiro, como mostram as métricas `numDeletionVectorsAdded` do histórico (seção 3).
- **Compactação automática.** `autoOptimize.autoCompact` e `optimizeWrite` nas tabelas da bronze (Figura 9).

```
%sql
SELECT version, timestamp, operation,
       operationParameters['zOrderBy'] AS zorder_por,
       operationMetrics['numRemovedFiles'] AS arquivos_removidos,
       operationMetrics['numAddedFiles'] AS arquivos_criados
FROM (DESCRIBE HISTORY ecommerce_lakehouse.auditoria.pedidos_snapshot)
WHERE operation = 'OPTIMIZE';
```

	version	timestamp	operation	zorder_por	arquivos_removidos	arquivos_criados
1	38	2026-10-01T01:02:05.000+00:...	OPTIMIZE	["cliente_id","pedido_ts"]	2	1
2	30	2026-09-29T02:53:13.000+00:...	OPTIMIZE	["cliente_id","pedido_ts"]	2	1
3	22	2026-09-29T02:44:05.000+00:...	OPTIMIZE	["cliente_id","pedido_ts"]	2	1
4	14	2026-09-29T02:35:19.000+00:...	OPTIMIZE	["cliente_id","pedido_ts"]	2	1
5	6	2026-09-29T02:25:24.000+00:...	OPTIMIZE	["cliente_id","pedido_ts"]	2	1

5 rows | 12.204s runtime | Refreshed at 10/5/26, 8:00 AM (Now)

This result is stored as `_sql_df` and can be used in other Python and SQL cells.

Figura 21 — Operações OPTIMIZE da tabela de auditoria: zOrderBy ["cliente_id","pedido_ts"] e arquivos removidos/criados.

```
%sql
SHOW TBLPROPERTIES ecommerce_lakehouse.silver.clientes;
SHOW TBLPROPERTIES ecommerce_lakehouse.silver.pedidos;
```

key	value
delta.feature.deletionVectors	supported
delta.feature.domainMetadata	supported
delta.feature.invariants	supported
delta.feature.rowTracking	supported
delta.minReaderVersion	3
delta.minWriterVersion	7
delta.parquet.compression.codec	zstd
delta.rowTracking.materializedRowCommitVersionColumnName	_row_commit_version-col-8ade81af-abac-4403-9fe6-30d4e27f989
delta.rowTracking.materializedRowIdColumnName	_row_id-col-88ed32c9-08c7-4a15-9c39-d60c90b40b8
io.unitycatalog.tableId	cfb65eeb-4ec1-4f75-b6bd-ef53191e64ce
pipeline_internal.catalogType	UNITY_CATALOG
pipeline_internal.enzymeMode	Advanced
pipelines.autoCdc.version	v2
pipelines.autoOptimize.zOrderCols	cliente_id,pedido_ts

Figura 22 — Propriedades de silver.pedidos: pipelines.autoOptimize.zOrderCols = cliente_id,pedido_ts e demais recursos Delta.

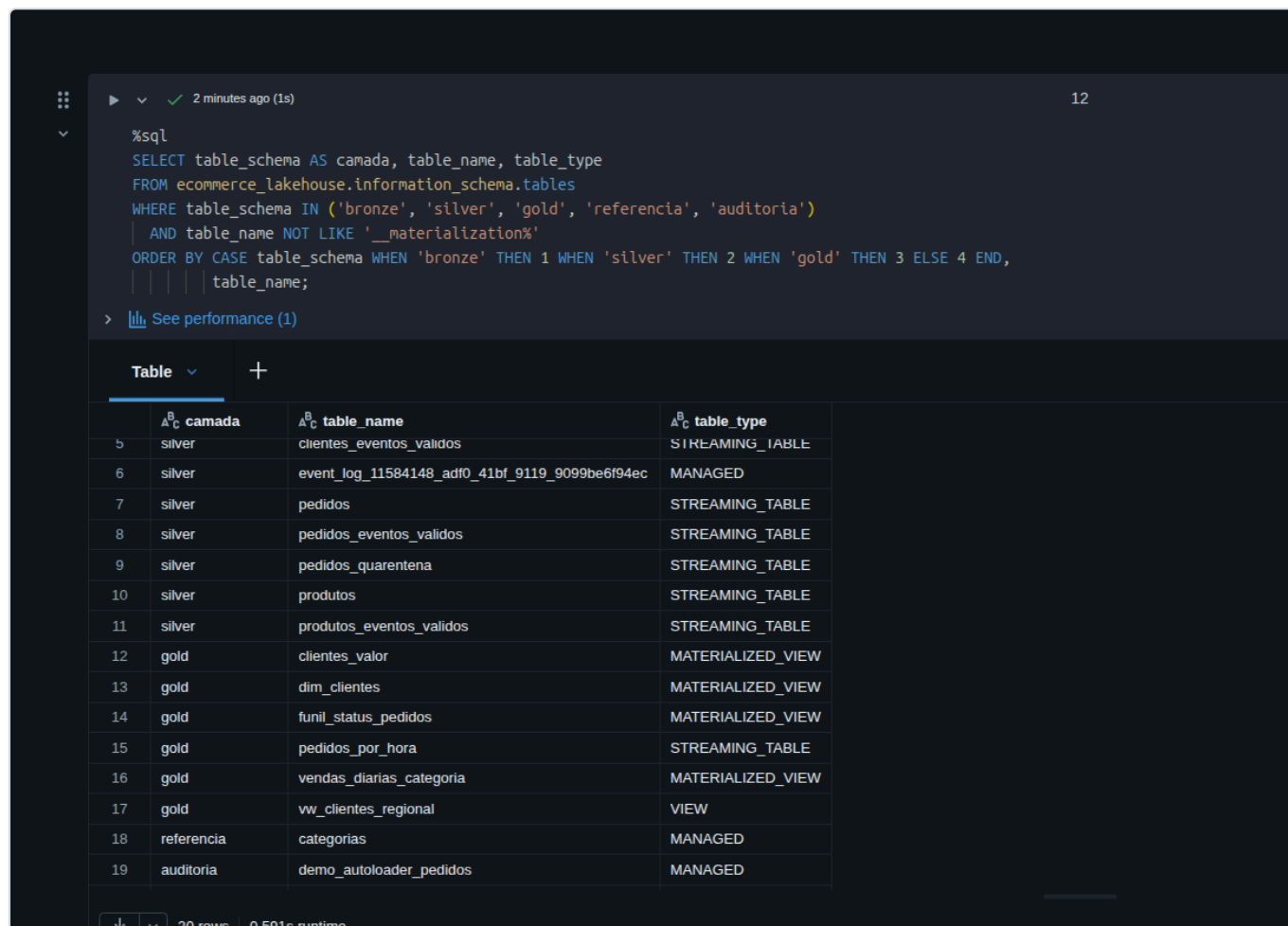
8.2 Dimensionamento automático (autoscale)

A Databricks Free Edition não permite criar clusters clássicos, apenas computação serverless. A computação serverless dimensiona os recursos de forma automática e transparente (o operador não escolhe o tamanho do cluster), o que atende à necessidade de acompanhar a demanda sem intervenção manual. Para registrar como o autoscale seria configurado num ambiente com clusters tradicionais, o arquivo docs/workflow/pipeline_classico_autoscale.json declara o mesmo pipeline com autoscale: {min_workers: 1, max_workers: 5, mode: ENHANCED}. Esse arquivo é apenas documental: não é referenciado pelo bundle e não foi executado.

9. Streaming, materialized views e views comuns

Os três tipos de dataset do Lakeflow são usados cada um onde faz sentido:

Tipo	Onde	Por quê
Streaming table	Bronze; eventos validados, quarentena e tabelas de CDC da silver; gold.pedidos_por_hora	Processamento incremental de fontes que só acrescentam dados
Materialized view	vendas_diarias_categoria, funil_status_pedidos, clientes_valor, dim_clientes	Agregações e junções recalculadas de forma incremental, com liquid clustering
View comum	gold.vw_clientes_regional; pedidos_enriquecidos (temporária)	A primeira aplica controle de acesso e deve ser avaliada em cada consulta, no contexto de quem consulta; a segunda é uma junção reaproveitada por três MVs



```
%sql
SELECT table_schema AS camada, table_name, table_type
FROM ecommerce_lakehouse.information_schema.tables
WHERE table_schema IN ('bronze', 'silver', 'gold', 'referencia', 'auditoria')
  AND table_name NOT LIKE '__materialization%'
ORDER BY CASE table_schema WHEN 'bronze' THEN 1 WHEN 'silver' THEN 2 WHEN 'gold' THEN 3 ELSE 4 END,
         table_name;
```

	camada	table_name	table_type
5	silver	clientes_eventos_validos	STREAMING_TABLE
6	silver	event_log_11584148_adf0_41bf_9119_9099be6f94ec	MANAGED
7	silver	pedidos	STREAMING_TABLE
8	silver	pedidos_eventos_validos	STREAMING_TABLE
9	silver	pedidos_quarentena	STREAMING_TABLE
10	silver	produtos	STREAMING_TABLE
11	silver	produtos_eventos_validos	STREAMING_TABLE
12	gold	clientes_valor	MATERIALIZED_VIEW
13	gold	dim_clientes	MATERIALIZED_VIEW
14	gold	funil_status_pedidos	MATERIALIZED_VIEW
15	gold	pedidos_por_hora	STREAMING_TABLE
16	gold	vendas_diarias_categoria	MATERIALIZED_VIEW
17	gold	vw_clientes_regional	VIEW
18	referencia	categorias	MANAGED
19	auditoria	demo_autoloader_pedidos	MANAGED

Figura 23 — Tabelas por camada com o tipo de cada uma (STREAMING_TABLE, MATERIALIZED_VIEW, VIEW e tabelas gerenciadas).

9.1 Watermark em streaming

gold.pedidos_por_hora agrega pedidos criados por janela de uma hora e canal, com WATERMARK pedido_ts DELAY OF INTERVAL 2 HOURS: tolera até duas horas de atraso antes de fechar uma janela, sem deixar o estado da agregação crescer indefinidamente. A fonte é silver.pedidos_eventos_validos (somente append), pois o Delta não permite fazer streaming a partir de uma tabela que sofre MERGE, caso de silver.pedidos (seção 13).



Figura 24 — gold.pedidos_por_hora: contagem e receita por janela de uma hora e canal.

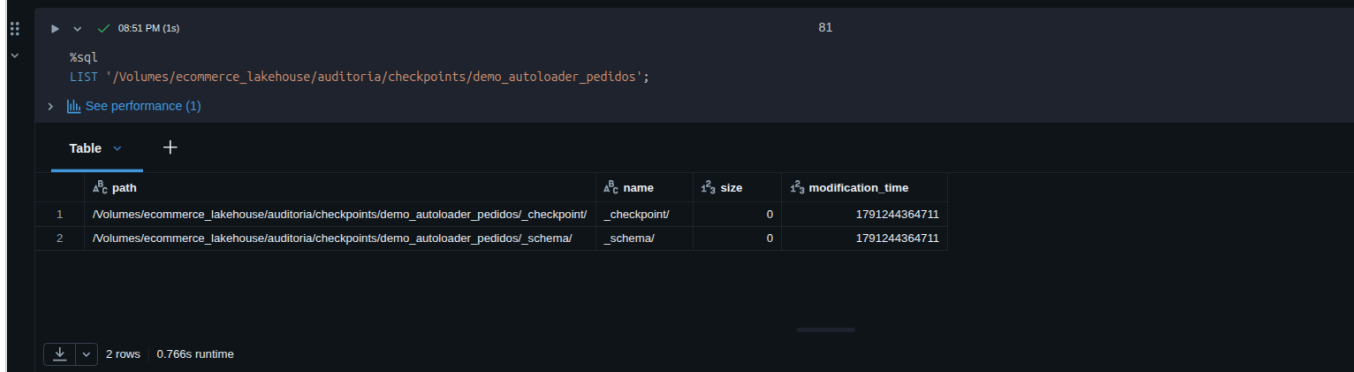
Observação. Algumas janelas aparecem com datas posteriores à execução (outubro de 2026). São pedidos com data futura injetados pelo gerador que a regra de qualidade não detecta (ver seção 14).

9.2 Checkpoint e deduplicação com watermark

Fora do pipeline declarativo, `src/operacao/demo_checkpoint_autoloader.py` usa o Auto Loader com Structured Streaming "clássico": `cloudFiles.schemaLocation`, `checkpointLocation` persistido em um volume dedicado (`auditoria.checkpoints`), `withWatermark` e `dropDuplicatesWithinWatermark`, com `trigger(availableNow=True)`. O checkpoint guarda o ponto de progresso, e uma nova execução continua do ponto em que parou, sem reprocessar.

12a · Checkpoint do Auto Loader

Lista os arquivos de checkpoint e de schema do Auto Loader standalone, que permitem retomar o processamento de onde parou.



```
%sql
LIST '/Volumes/ecommerce_lakehouse/auditoria/checkpoints/demo_autoloader_pedidos';
```

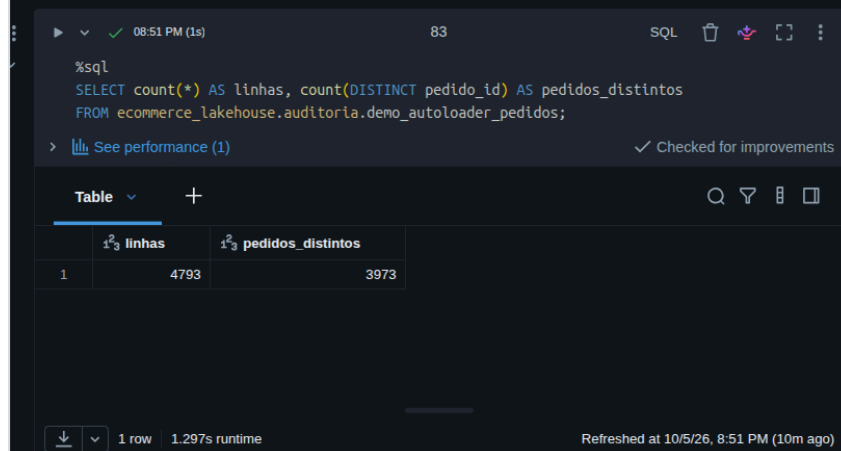
	path	name	size	modification_time
1	/Volumes/ecommerce_lakehouse/auditoria/checkpoints/demo_autoloader_pedidos_checkpoint/	_checkpoint/	0	1791244364711
2	/Volumes/ecommerce_lakehouse/auditoria/checkpoints/demo_autoloader_pedidos_schema/	_schema/	0	1791244364711

Figura 25 — Pastas `_checkpoint` e `_schema` do Auto Loader standalone no volume `auditoria.checkpoints`.

A tabela do demo contém 4.793 linhas para 3.973 pedidos distintos. A diferença não é duplicidade falha: a chave de deduplicação é `(pedido_id, evento_ts)`, então eventos do mesmo pedido em instantes diferentes (as mudanças de status) são mantidos, e só repetições do mesmo evento são removidas.

12b · Deduplicação com watermark

Compara as linhas da tabela do demo com os pedidos distintos, mostrando o efeito do `dropDuplicatesWithinWatermark`.



```
%sql
SELECT count(*) AS linhas, count(DISTINCT pedido_id) AS pedidos_distintos
FROM ecommerce_lakehouse.auditoria.demo_autoloader_pedidos;
```

	linhas	pedidos_distintos
1	4793	3973

Figura 26 — Linhas e pedidos distintos da tabela `auditoria.demo_autoloader_pedidos`.

10. Dependência externa e publicação dos datasets

A gold depende de uma tabela mantida **fora** do pipeline: `referencia.categorias` (8 categorias com margem-alvo), carregada por um `MERGE` no notebook de setup, com chave primária informativa e `CHECK` de margem entre 0 e 1, e apenas lida pelo Lakeflow. Ela simula a integração com um sistema de referência mantido por outra equipe. A view temporária `pedidos_enriquecidos` junta `silver.pedidos`, `silver.produtos` e essa tabela antes de alimentar três materialized views, e a linhagem (seção 7.4) torna a dependência explícita e rastreável.

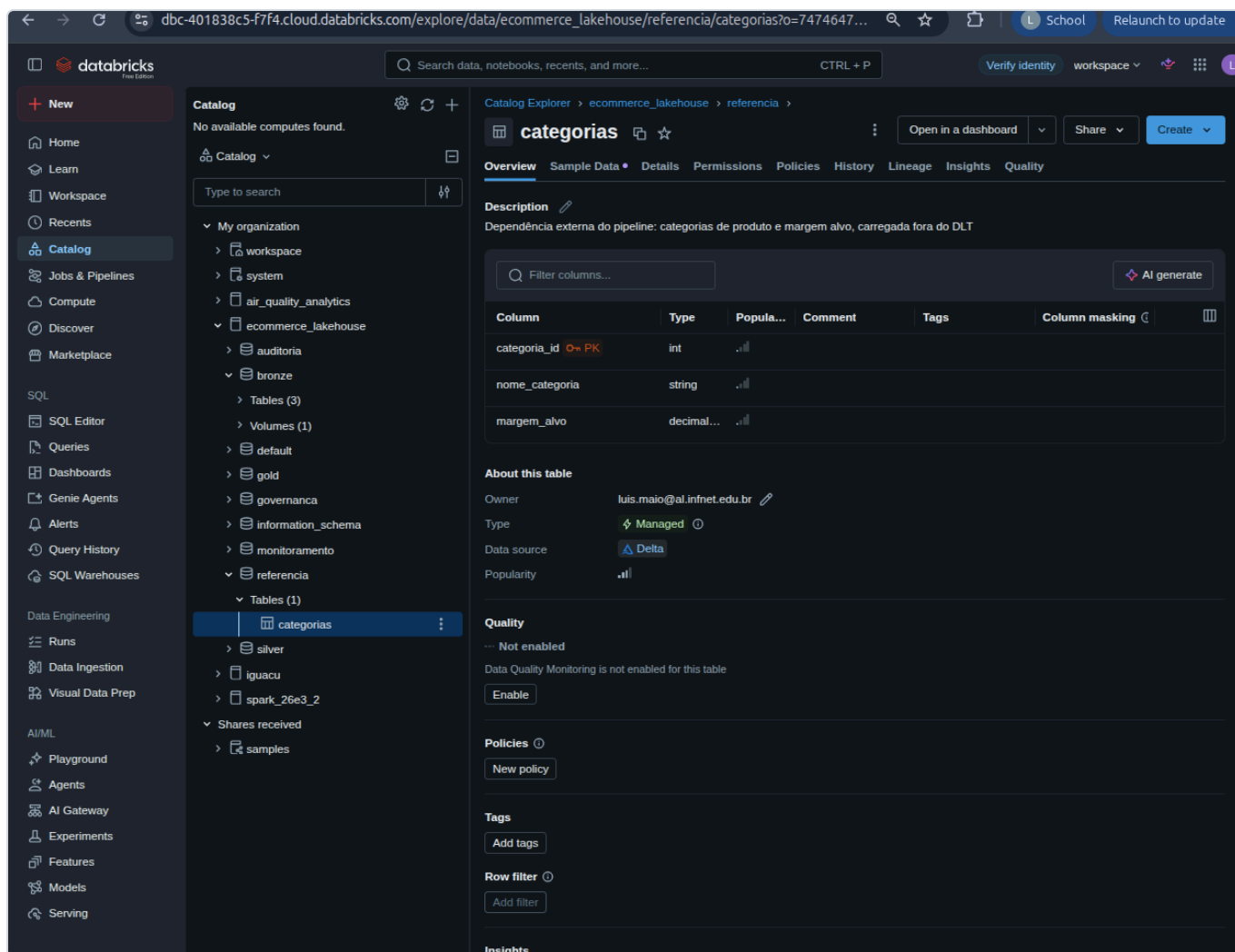


Figura 27 — Tabela referencia.categorias no Catalog Explorer: chave primária, tipo Managed/Delta e descrição de dependência externa.

11. Manutenção e operação

A manutenção das tabelas Delta está em `src/operacao/manutencao_delta.sql`, executada como tarefa do job principal e também como o job agendado `manutencao` (cron das 02:30):

- `OPTIMIZE ... ZORDER BY`: compacta arquivos pequenos e ordena fisicamente os dados (seção 8).
- `REORG TABLE ... APPLY (PURGE)`: remove fisicamente as linhas marcadas por deletion vectors.
- `VACUUM ... RETAIN 168 HOURS`: apaga arquivos obsoletos fora da janela de retenção (executado primeiro em `DRY RUN`).

As versões 6, 7 e 8 do histórico (Figura 5) registram a sequência `OPTIMIZE`, `VACUUM START` e `VACUUM END`. A retenção de 168 horas preserva o time travel recente: o `VACUUM` não apagou arquivos naquela execução porque nenhum estava elegível.

03 · Execução dos flows do pipeline

Mostra, a partir do event log, o status final de cada flow do pipeline em cada atualização.

▶ 3 minutes ago (2s) 17

```
%sql
SELECT timestamp,
       origin.update_id AS update_id,
       origin.flow_name AS flow,
       details:flow_progress.status::STRING AS status
FROM ecommerce_lakehouse.monitoramento.vw_event_log
WHERE event_type = 'flow_progress'
      AND details:flow_progress.status::STRING IN ('COMPLETED', 'FAILED')
ORDER BY timestamp DESC
LIMIT 40;
```

> See performance (1)

Table +

	timestamp	update_id	flow	status
1	2026-10-05T23:44:59.581+00:...	c95d954f-d204-4364-8b85-54cceac6a74f	ecommerce_lakehouse.gold.vendas_diarias_categoria	COMPLETED
2	2026-10-05T23:44:59.545+00:...	c95d954f-d204-4364-8b85-54cceac6a74f	ecommerce_lakehouse.gold.clientes_valor	COMPLETED
3	2026-10-05T23:44:59.472+00:...	c95d954f-d204-4364-8b85-54cceac6a74f	ecommerce_lakehouse.gold.dim_clientes	COMPLETED
4	2026-10-05T23:44:59.427+00:...	c95d954f-d204-4364-8b85-54cceac6a74f	ecommerce_lakehouse.gold.funil_status_pedidos	COMPLETED
5	2026-10-05T23:44:40.535+00:...	c95d954f-d204-4364-8b85-54cceac6a74f	ecommerce_lakehouse.silver.pedidos_cdc_flow	COMPLETED
6	2026-10-05T23:44:39.866+00:...	c95d954f-d204-4364-8b85-54cceac6a74f	ecommerce_lakehouse.gold.pedidos_por_hora	COMPLETED
7	2026-10-05T23:44:37.812+00:...	c95d954f-d204-4364-8b85-54cceac6a74f	ecommerce_lakehouse.silver.clientes_cdc_flow	COMPLETED
8	2026-10-05T23:44:37.221+00:...	c95d954f-d204-4364-8b85-54cceac6a74f	ecommerce_lakehouse.silver.produtos_cdc_flow	COMPLETED

Figura 28 — Event log: status COMPLETED de cada flow na atualização mais recente do pipeline.

12. Métricas do pipeline e impacto das otimizações

12.1 Volume por tabela

Tabela	Linhas	Tabela	Linhas
bronze.clientes_raw	571	silver.pedidos_quarentena	154
bronze.pedidos_raw	4.800	gold.vendas_diarias_categoria	1.833
bronze.produtos_raw	46	gold.funil_status_pedidos	5
silver.clientes_eventos_validos	554	gold.clientes_valor	476
silver.pedidos_eventos_validos	4.646	gold.dim_clientes	476
silver.clientes	544	gold.pedidos_por_hora	2.786
silver.pedidos	3.873	referencia.categorias	8
silver.produtos	39		

A progressão mostra o refinamento: a bronze guarda 4.800 eventos de pedido, a silver aceita 4.646 e mantém 3.873 pedidos únicos (as 800 mudanças de status atualizam linhas existentes), e a gold resume isso em 5 estados do funil e 1.833 combinações de dia e categoria.

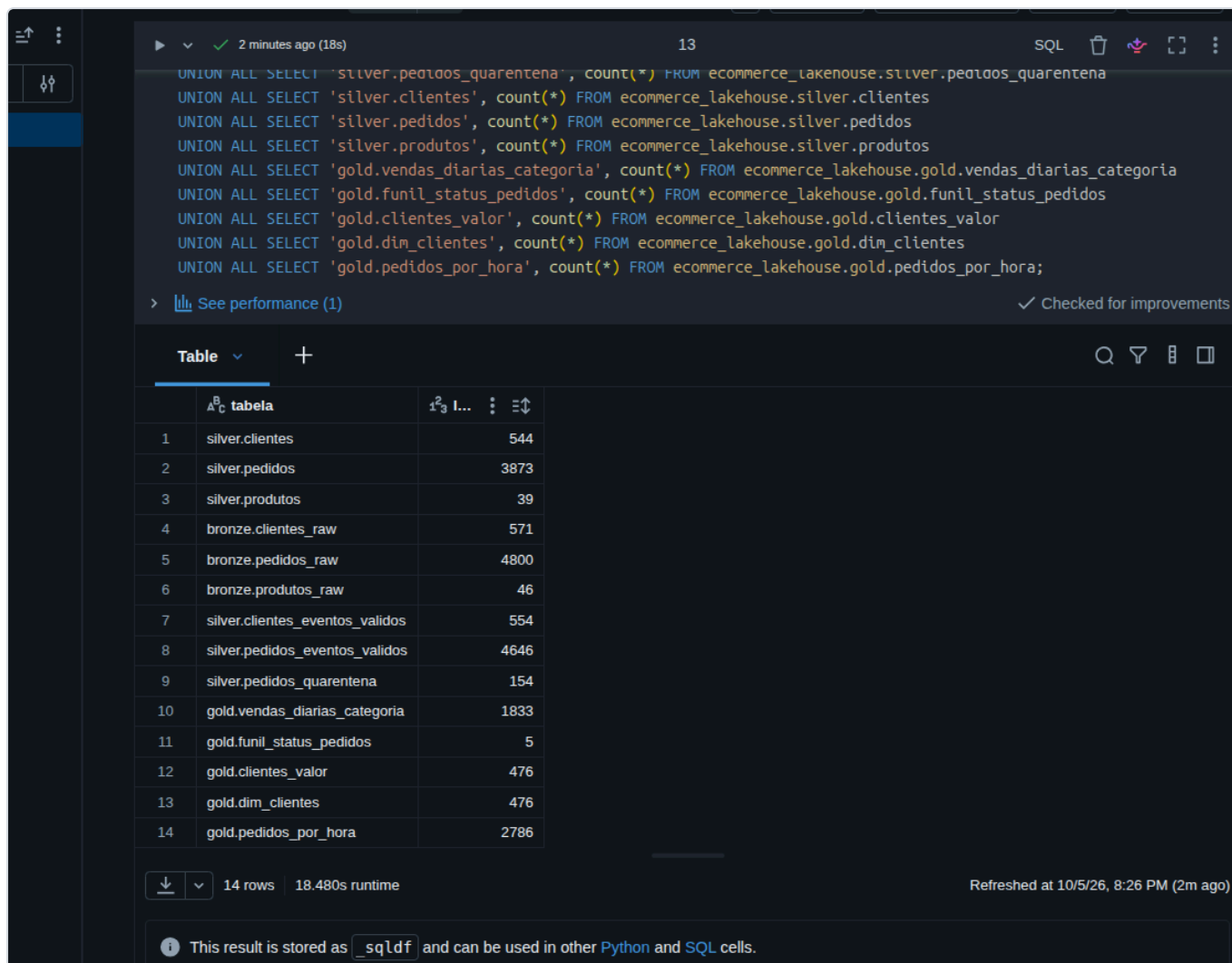


Figura 29 — Contagem de linhas das tabelas das três camadas.

12.2 Linhas processadas e descartadas

Flow	Linhas processadas	Descartadas pelas expectations
bronze.clientes_raw / pedidos_raw / produtos_raw	571 / 4.800 / 46	0
silver.clientes_eventos_validos	554	17
silver.pedidos_eventos_validos	4.646	154
silver.produtos_eventos_validos	46	0
silver.pedidos_quarentena	154	0

As expectations descartaram 17 eventos de clientes (3,0% de 571) e 154 de pedidos (3,2% de 4.800), coerentes com os defeitos injetados pelo gerador (cerca de 3% e 4%). Os 154 descartados de pedidos são os mesmos 154 que a quarentena guarda: o descarte não é perda, é redirecionamento. Os fluxos da gold somam mais linhas processadas do que existem nas tabelas porque as materialized views são reavaliadas a cada atualização.

15a · Linhas processadas e descartadas

Soma, por flow do pipeline, as linhas processadas e as descartadas pelas expectations.

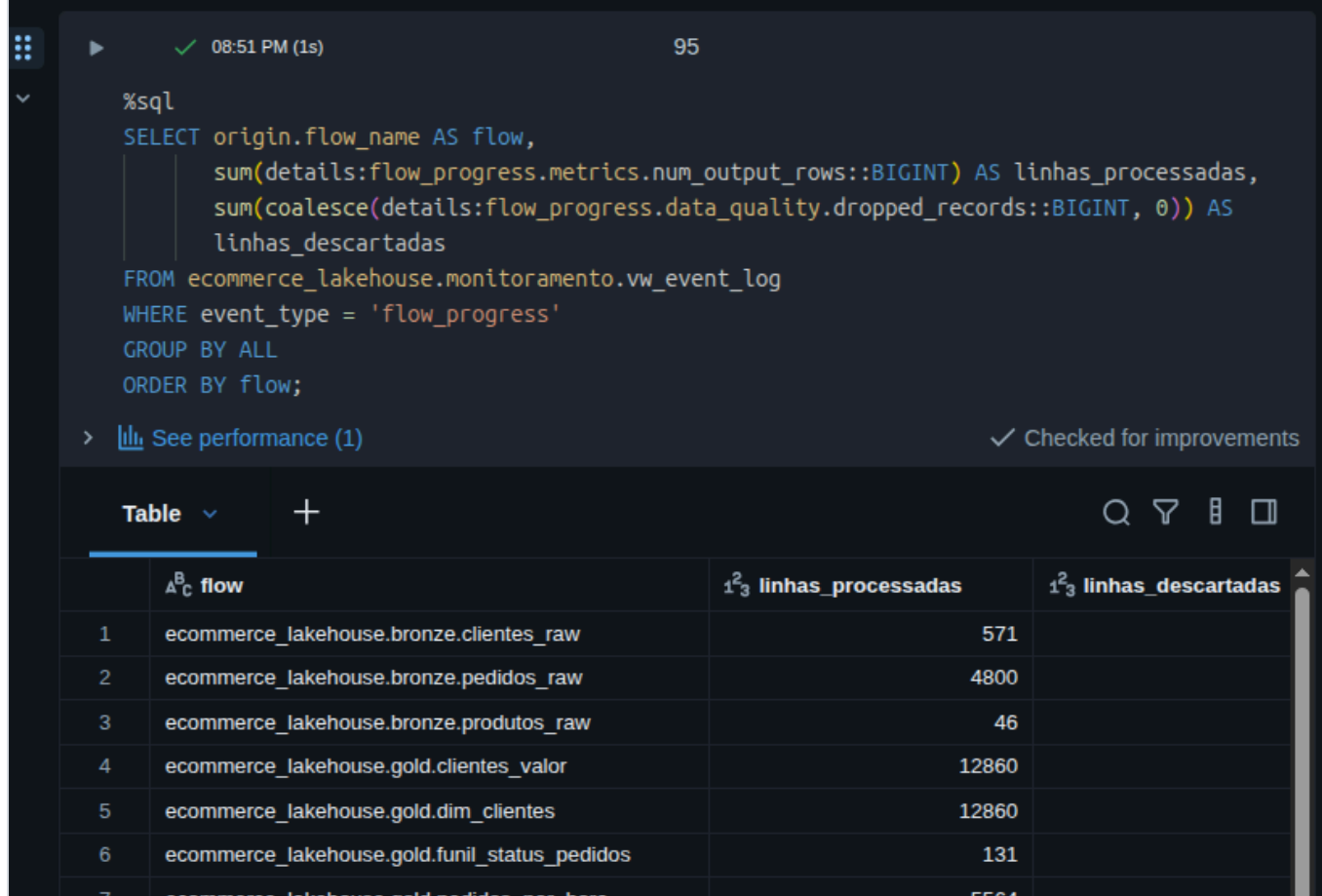


Figura 30 — Linhas processadas por flow, extraídas do event log (trecho inicial da tabela).

12.3 Duração das atualizações

Início da atualização (UTC)	Contexto	Duração
2026-10-01 00:54	Job completo, atualização do lote 1	177,5 s
2026-10-01 00:58	Job completo, atualização do lote 2	129,3 s
2026-10-05 23:43	Reexecução do pipeline, sem arquivos novos	118,6 s

As durações ficam na mesma ordem de grandeza (cerca de 2 a 3 minutos) apesar de o lote 1 processar bem mais dados que o lote 2 e de a reexecução não ter dados novos. Isso indica que, nesse volume (milhares de linhas), o tempo é dominado pelo custo fixo de provisionar a computação serverless e planejar o grafo, e não pela quantidade de dados. A otimização relevante aqui é o processamento **incremental**: o Auto Loader lê só arquivos novos e as materialized views usam atualização incremental quando possível, de modo que o custo cresce com o dado novo, e não com o histórico. Em volumes maiores, Z-Order e liquid clustering passam a reduzir a quantidade de arquivos lidos nas consultas (data skipping), efeito que este volume não permite medir.

15b · Duração das atualizações

Mostra o início, o fim e a duração em segundos de cada atualização do pipeline.

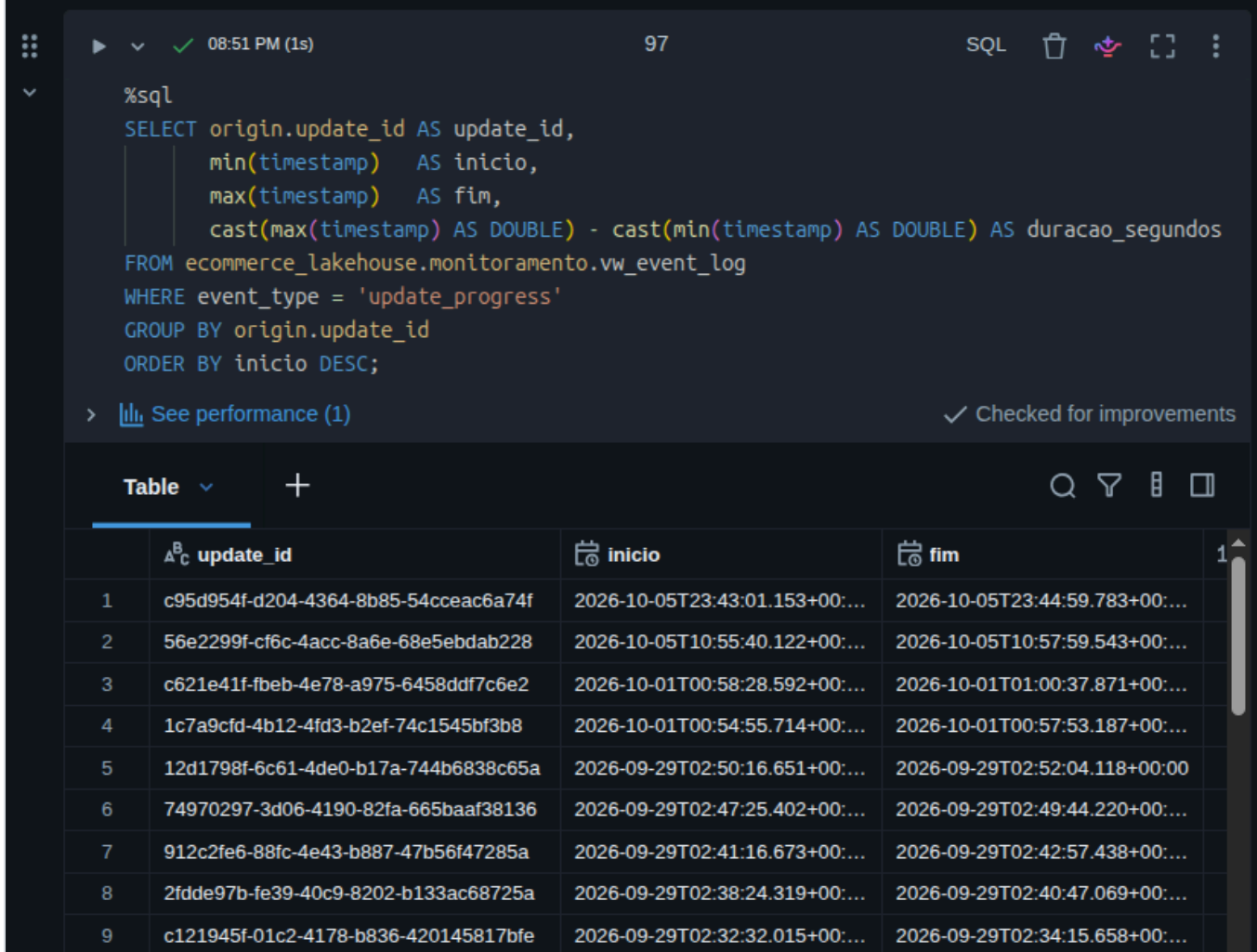


Figura 31 — Início, fim e duração de cada atualização do pipeline (trecho do event log).

12.4 Compactação física

Na tabela de auditoria, o `OPTIMIZE ... ZORDER BY` transformou 2 arquivos (56.389 bytes) em 1 arquivo (51.125 bytes), e removeu o deletion vector pendente. É a correção do *small file problem*: cada `UPDATE`, `DELETE` ou `MERGE` tende a gerar arquivos pequenos, e sem compactação o número de arquivos cresce a cada ciclo e degrada a leitura. O ganho absoluto aqui é pequeno (uma tabela de milhares de linhas), mas a técnica é a mesma que se aplicaria às tabelas de CDC em produção.

13. Adaptações técnicas durante a implementação

O pipeline foi executado de verdade, e isso expôs comportamentos do motor que exigiram correções. Em todos os casos a intenção do requisito foi mantida.

#	Sintoma	Causa	Correção
1	PRINCIPAL_DOES_NOT_EXIST no GRANT	Grupos criados pela CLI eram de workspace	Recriados como grupos de conta
2	Erro de sintaxe em CHECK	Constraint inline não aceita no CREATE TABLE	ALTER TABLE ... ADD CONSTRAINT
3	Erro de sintaxe em ROW FILTER	Não suportado em materialized view nesta versão	Filtro numa view comum (vw_clientes_regional)
4	UNRESOLVED_ROUTINE no MASK	Catálogo padrão da MV diferente do do pipeline	Funções com nome totalmente qualificado
5	Erro de sintaxe em WATERMARK	Cláusula depois do WHERE	Logo após o FROM STREAM(...)
6	DELTA_SOURCE_TABLE_IGNORE_CHANGES	Streaming lendo de uma tabela que sofre MERGE	Fonte trocada para a tabela de eventos validados (só append)
7	Schema incompatível em dim_clientes	__START_AT herda o tipo do SEQUENCE BY	Coluna vigente_desde_evento_seq BIGINT
8	DELTA_CDC_NON_CONSTANT_ARGUMENT	table_changes e RESTORE exigem literal	Versão calculada em célula Python e interpolada
9	Campo event_log desconhecido no bundle	A CLI usada não suporta publicar o event log	Consultas com event_log(TABLE(...))
10	UNRESOLVED_COLUMN expectation.name	LATERAL explode(...) AS x nomeia a relação, não a coluna	LATERAL VIEW explode(...) AS x

14. Limitações conhecidas

Estas são as limitações identificadas ao revisar o projeto. Nenhuma impede a demonstração dos requisitos, mas devem ser conhecidas por quem for evoluir a solução.

- Regra de data coerente.** Compara pedido_ts com evento_ts, e o gerador grava o mesmo valor nos dois campos; a regra nunca reprova (0 de 4.800). Consequência medida: 16 dos 3.873 pedidos de silver.pedidos têm data posterior à execução e chegam à gold (visível em pedidos_por_hora). Correção sugerida: comparar com current_timestamp().
- Quarentena apenas para pedidos.** Clientes inválidos (17 eventos) são descartados sem ficar isolados. Uma clientes_quarentena seguiria o mesmo padrão.
- Filtro por UF numa view.** Como o ROW FILTER não vale em materialized view, a restrição por região só existe em gold.vw_clientes_regional. Os analistas têm SELECT em todo o schema gold, portanto podem consultar gold.dim_clientes diretamente (com PII mascarada, mas sem filtro de UF). Um próximo passo é conceder apenas a view aos analistas.
- Filtro por UF não validado com usuário analista.** O grupo ecom_analistas não tem membros, e o efeito do filtro (e a resolução do parâmetro uf dentro da função) não foi testado de ponta a ponta. A máscara foi verificada (Figura 18).
- Autoscale clássico não executado.** Só há configuração de referência (seção 8.2); a execução real foi em serverless.

6. **Jobs de ingestão e manutenção pausados** por escolha, para não consumir computação sem uso real.
7. **Dados sintéticos e volume pequeno.** As conclusões sobre desempenho em escala (seção 12) são qualitativas.

15. Atendimento aos requisitos

Requisito	Seção	Evidência	Situação
Anatomia da tabela Delta e protocolo de transação	3	Figuras de DESCRIBE DETAIL/HISTORY e propriedades	Atendido
Separação entre bronze, silver e gold	2, 12	Diagrama, tabelas por camada, contagens	Atendido
Framework Delta Live Tables (fluxos e dependências)	2.2	Grafo do pipeline com 16 flows	Atendido
Captura escalável de novos arquivos	4	Linhas por arquivo de origem, lotes 1 e 2	Atendido
Regras de validação e isolamento de dados com falha	5	Expectations, quarentena, reconciliação $4.800 = 4.646 + 154$	Atendido (ressalvas 1 e 2 da seção 14)
CDC refletindo as mudanças	6	SCD2 com versões, $544 = 476 + 68$, evento atrasado, SCD1	Atendido
Federação de identidades e catalogação centralizada	7.1, 7.4	Grupos de conta, GRANTS, linhagem	Atendido
Mascaramento ou isolamento de datasets sensíveis	7.2, 7.3	column_masks, consulta mascarada, isolamento por GRANT	Atendido com ressalvas (3 e 4 da seção 14)
Z-Order, deletion vectors ou ordenação avançada	8.1	Histórico de OPTIMIZE, propriedades, métricas de deletion vectors	Atendido
Dimensionamento automático (autoscale)	8.2	Serverless; JSON de referência	Atendido com ressalva (5 da seção 14)
Versionamento Delta para auditoria e recuperação	3	DELETE acidental seguido de RESTORE	Atendido
Checkpoints e marcas d'água	9.1, 9.2	pedidos_por_hora, checkpoint do Auto Loader, deduplicação	Atendido
Streaming tables, materialized views e views comuns	9	Tabela de tipos e lista do Catalog	Atendido
Datasets finais com dependências externas	10	referencia.categorias e linhagem	Atendido
Manutenção de tabelas (compactação e vacuum)	11	OPTIMIZE, REORG, VACUUM no histórico; job agendado	Atendido (job agendado pausado)
Interpretação das métricas do pipeline	12	Event log: qualidade, linhas, duração	Atendido

16. Conclusão

O projeto entrega um Data Lakehouse funcional no padrão Medallion, executado de ponta a ponta num workspace Databricks: ingestão incremental com Auto Loader, pipeline declarativo com 16 flows, qualidade de dados com isolamento explícito dos registros rejeitados, CDC com estratégias SCD1 e SCD2 adequadas a cada entidade, governança centralizada no Unity Catalog com grupos de conta, máscaras de PII e linhagem, e otimizações de armazenamento (Z-Order, liquid clustering, deletion vectors, compactação). As garantias transacionais do Delta foram demonstradas com time travel e recuperação de um estado anterior.

As limitações do ambiente gratuito (computação exclusivamente serverless) e do motor (sem ROW FILTER em materialized view) foram tratadas e documentadas, e os pontos fracos que a revisão identificou estão declarados na seção 14, com a correção sugerida para cada um, em vez de omitidos.

Repositório e reprodutibilidade

Código, jobs, notebooks operacionais, notebook de validação e README com as instruções de execução: github.com/LuisFelipeMTCS/ecommerce_lakehouse. A publicação e a execução reproduzem-se com `databricks bundle validate`, `databricks bundle deploy` e `databricks bundle run setup_e_carga`.